

Informe técnico integral

Backend con Django RF y pasarelas de pago (Stripe, PayPal, Redsys)

Sumario

1. ¿Qué es este proyecto y para qué sirve?.....	3
2. Las tecnologías utilizadas y por qué.....	4
2.1 Python.....	4
2.2 Django.....	4
2.3 Django REST Framework (DRF).....	5
2.4 SQLite.....	5
2.5 stripe (SDK oficial de Stripe).....	5
2.6 paypalrestsdk.....	6
2.7 pyredsys.....	6
2.8 python-dotenv.....	6
2.9 requests.....	6
2.10 Bootstrap 5.....	7
2.11 Stripe CLI.....	7
2.12 ngrok.....	7
2.13 flake8.....	7
3. Instalación paso a paso.....	8
3.1 Requisitos previos.....	8
3.2 Clonar el repositorio.....	8
3.3 Crear el entorno virtual.....	8
3.4 Instalar las librerías.....	9
3.5 Crear el archivo .env.....	10
3.6 Configurar settings.py.....	12
3.7 Aplicar las migraciones.....	14
3.8 Crear el superusuario.....	14
3.9 Arrancar el servidor.....	15
3.10 Servicios adicionales necesarios.....	15
4. La estructura del proyecto en detalle.....	16
4.1 La raíz del proyecto.....	16
4.2 La carpeta config/.....	17
4.3 La carpeta core/.....	19
4.4 La carpeta frontend/.....	21
4.5 La carpeta docs/.....	22
4.6 La carpeta backlog/.....	22
4.7 Los archivos de migración en detalle.....	23
5. Los modelos. La estructura de la base de datos.....	24
5.1 ¿Qué es un modelo Django?.....	24
5.2 El modelo Provider (Proveedor).....	24
5.3 El modelo Transaction (Transacción).....	26
5.4 La relación entre los modelos.....	28
5.5 El ORM de Django: consultas sin SQL.....	28
6. Los serializers. Validación y conversión de datos.....	30
6.1 ¿Qué es un serializer y para qué sirve?.....	30
6.2 El ProviderSerializer.....	30
6.3 El TransactionSerializer.....	31
6.4 La validación del importe.....	32
6.5 La validación cruzada de campos.....	33
6.6 Cómo funciona la serialización en la práctica.....	33
6.7 Por qué los importes son strings en el JSON.....	34

7. Las vistas de la API. La lógica principal.....	35
7.1 Los imports, qué librerías se usan y por qué.....	35
7.2 El ProviderViewSet.....	37
7.3 El TransactionViewSet, la parte básica.....	38
7.4 El método get_queryset. Filtros personalizados.....	39
7.5 El método create. Integración con Stripe al crear transacciones.....	40
7.6 El webhook de Stripe.....	42
7.7 El formulario de pago de Stripe.....	44
7.8 El pago con Redsys.....	45
7.9 El webhook de Redsys.....	47
7.10 El pago con PayPal.....	48
7.11 La ejecución del pago PayPal.....	50
8. Las vistas del frontend. El panel de gestión.....	51
8.1 La arquitectura del frontend.....	51
8.2 Los imports y la configuración base.....	51
8.3 La función auxiliar get_token.....	52
8.4 La vista login.....	53
8.5 La vista logout.....	54
8.6 La vista dashboard.....	55
8.7 La vista providers.....	56
8.8 La vista transactions, la más compleja.....	57
8.9 Las vistas update_transaction y delete_provider.....	59
9. Las plantillas HTML: El aspecto visual.....	60
9.1 ¿Qué es el sistema de plantillas de Django?.....	60
9.2 La plantilla base: base.html.....	61
9.3 La plantilla de login: login.html.....	63
9.4 La plantilla del dashboard: dashboard.html.....	65
9.5 La plantilla de transacciones: transactions.html.....	67
9.6 Las plantillas de pago.....	72
10. Configuración y uso de cada pasarela de pago.....	74
10.1 Stripe: Configuración completa.....	74
10.2 Redsys: Configuración completa.....	77
10.3 PayPal: Configuración completa.....	80
10.4 Comparativa de las tres pasarelas.....	82
11. Los tests automáticos.....	83
11.1 ¿Por qué son importantes los tests?.....	83
11.2 Los tests de la API: core/tests.py.....	83
11.3 Los tests del frontend: frontend/tests.py.....	87
11.4 Cómo ejecutar los tests.....	89
11.5 El verificador de estilo: flake8.....	89
12. Git y el control de versiones.....	90
12.1 ¿Qué es Git y por qué es importante?.....	90
12.2 Conceptos fundamentales de Git.....	90
12.3 El archivo .gitignore.....	91
12.4 El flujo de trabajo del proyecto.....	92
12.5 Autenticación con GitHub: Personal Access Token.....	93
12.6 El historial de commits del proyecto.....	94
12.7 Trabajar en múltiples equipos.....	94
12.8 Comandos Git más usados en el proyecto.....	95
12.9 Por qué el requirements.txt es flexible.....	95
13. Resumen final.....	96

1. ¿Qué es este proyecto y para qué sirve?

Imagina que tienes una tienda online y quieres que tus clientes puedan pagar. Tienes varias opciones: **Stripe** (muy popular en Europa y EEUU), **PayPal** (conocido mundialmente) y **Redsys** (la pasarela de los bancos españoles como BBVA, CaixaBank o Santander).

El problema es que cada pasarela funciona de forma diferente, tiene su propia API, sus propias claves y su propia forma de notificarte cuando se completa un pago. Gestionar todo eso en cada tienda que tengas sería un caos.

Este proyecto resuelve ese problema: es un backend centralizado que:

- Registra qué pasarelas tienes configuradas (Stripe, PayPal, Redsys).
- Guarda cada intento de pago con su estado (pendiente, completado, fallido).
- Se comunica con cada pasarela para procesar los pagos.
- Recibe notificaciones automáticas cuando un pago se completa o falla.
- Gestiona incidencias como impagos, errores de conexión o devoluciones.
- Ofrece un panel web visual para gestionar todo.

Técnicamente es una **API REST**, es decir, un servicio web que otros sistemas pueden consultar enviando peticiones **HTTP** estándar (**GET** para consultar, **POST** para crear, **PUT** para modificar, **DELETE** para eliminar).

2. Las tecnologías utilizadas y por qué.

Antes de instalar nada, es importante entender qué es cada cosa y por qué se eligió. Aquí va una explicación desde cero.

2.1 Python.

Python es el lenguaje de programación en el que está escrito todo el proyecto. Es el idioma que habla el código.

Se eligió porque:

- Es muy legible: el código se parece casi al inglés normal.
- Es el más usado en desarrollo web backend junto con JavaScript.
- El enunciado del proyecto lo exigía explícitamente.
- Tiene librerías para casi todo, incluyendo las tres pasarelas de pago.

La versión usada es **Python 3.12**. Importante: **Django 6.0.4** requiere Python 3.12. En Python 3.10 se usa Django 5.2.

2.2 Django.

Django es un **framework web** para Python. Un framework es como un kit de construcción que ya tiene resueltas las partes comunes de cualquier aplicación web:

- Sistema de base de datos (ORM, Object Relational Mapper).
- Sistema de URLs y enrutamiento.
- Panel de administración automático.
- Sistema de autenticación y usuarios.
- Protección contra ataques comunes (CSRF, SQL injection...).
- Sistema de plantillas HTML.

Sin Django tendríamos que escribir todo eso desde cero. Con Django solo escribimos la lógica específica de nuestro proyecto.

La versión usada es **Django 6.0.4**.

2.3 Django REST Framework (DRF).

Django por sí solo está pensado para hacer webs tradicionales con páginas HTML. **Django REST Framework** es una extensión que añade las herramientas necesarias para construir **APIs REST**.

¿Qué es una API REST? Es un servicio web que responde a peticiones HTTP con datos en formato **JSON**. Por ejemplo:

```
GET /api/transactions/  
-> [{"id": 1, "amount": "99.99", "status": "pending"}, ...]
```

DRF nos proporciona:

- **Serializers**: para validar y convertir datos.
- **ViewSet**s: para gestionar automáticamente GET, POST, PUT, DELETE.
- **Autenticación por token**: para proteger los **endpoints**.
- **Interfaz navegable**: para probar la API desde el navegador.

La versión usada es **DRF 3.17.1**.

2.4 SQLite.

SQLite es la base de datos del proyecto. Una base de datos es donde se guarda toda la información de forma permanente: los proveedores, las transacciones, los usuarios, ...

SQLite es especial porque **no es un servidor**, es simplemente un archivo en el disco (**db.sqlite3**). Esto lo hace ideal para desarrollo porque no hay que instalar ni configurar nada extra.

En un proyecto en producción real se usaría **PostgreSQL** o **MySQL**, que son más potentes y soportan múltiples conexiones simultáneas.

2.5 stripe (SDK oficial de Stripe).

Es la librería oficial de Stripe para Python. Un SDK (Software Development Kit) es un conjunto de herramientas que nos permite comunicarnos con la API de Stripe sin tener que hacer las peticiones HTTP manualmente.

Con esta librería podemos:

- Crear **PaymentIntents** (intenciones de pago).
- Verificar **webhooks** (notificaciones de Stripe).
- Gestionar **reembolsos**.

La versión usada es **stripe 15.1.0**.

2.6 paypalrestsdk.

Es la librería oficial de PayPal para Python. Funciona de forma similar al SDK de Stripe pero con la filosofía de PayPal: crear un pago, redirigir al usuario a PayPal para que lo apruebe y ejecutar el pago tras la aprobación.

La versión usada es **paypalrestsdk 1.13.3**.

2.7 pyredsys.

Es una librería Python para trabajar con Redsys. A diferencia de Stripe y PayPal, Redsys no tiene un SDK oficial moderno para Python, así que usamos esta librería de terceros.

Nos permite:

- Generar los parámetros del pago en el formato que exige Redsys.
- Firmar los parámetros con HMAC-SHA256 (el sistema de seguridad de Redsys).
- Verificar las notificaciones que Redsys nos envía.

La versión usada es **pyredsys 0.1.6**.

2.8 python-dotenv.

Esta librería lee el archivo **.env** y carga su contenido como variables de entorno.

¿Por qué es importante? Porque las claves secretas (de Stripe, PayPal, Redsys) **nunca deben estar escritas directamente en el código**. Si alguien accede al repositorio de GitHub no debe poder ver esas claves. Con python-dotenv las guardamos en un archivo **.env** que está en **.gitignore** y nunca se sube.

La versión usada es **python-dotenv 1.2.2**.

2.9 requests.

Es la librería más popular de Python para hacer peticiones HTTP. La usamos en el **frontend** para que las **vistas** Django llamen a la propia API REST del backend.

¿Por qué el frontend llama a su propia API? Porque el frontend está construido con **Django Templates** y necesita los datos en formato JSON para luego renderizarlos en HTML. Esta arquitectura permite que el frontend y el backend sean completamente independientes.

2.10 Bootstrap 5.

Bootstrap es una librería **CSS** (de estilos visuales) que se carga desde internet via **CDN**. Nos permite hacer el frontend visualmente atractivo y responsive sin escribir CSS desde cero.

Se importa directamente en los templates HTML:

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
```

2.11 Stripe CLI.

Es una herramienta de línea de comandos oficial de Stripe. Durante el desarrollo, cuando Stripe quiere notificarnos que un pago se completó, necesita enviar una petición HTTP a nuestro servidor. El problema es que nuestro servidor está en **localhost** y Stripe no puede llegar ahí desde internet.

La Stripe CLI soluciona esto creando un túnel que **reenvía los webhooks de Stripe a nuestro servidor local**:

```
Stripe → Stripe CLI → http://127.0.0.1:8000/api/webhooks/stripe/
```

2.12 ngrok.

ngrok es similar a la Stripe CLI pero genérico. Crea un **túnel HTTPS público** que expone nuestro servidor local a internet. Lo usamos para Redsys porque Redsys necesita una URL pública real para enviar las notificaciones de pago.

```
https://xxxx.ngrok-free.dev → http://127.0.0.1:8000
```

2.13 flake8.

Es una herramienta que comprueba que el código sigue las normas de estilo PEP8 (el estándar oficial de Python). PEP8 define reglas como la longitud máxima de línea (79 caracteres), los espacios entre funciones, etc.

Se ejecuta con:

```
flake8 core/ --exclude=core/migrations/
```

Si no hay errores, el código cumple el estándar.

3. Instalación paso a paso.

Vamos a explicar todo lo que hay que hacer para tener el proyecto funcionando en un ordenador nuevo, desde cero.

3.1 Requisitos previos.

Antes de empezar necesitas tener instalado en tu sistema:

Git: para descargar el proyecto desde GitHub. Comprueba si lo tienes con:

```
git --version
```

Python: el lenguaje del proyecto. Comprueba si lo tienes con:

```
python3 --version
```

Necesitas Python 3.10 o superior. Si tienes 3.12 mejor, porque es la versión con la que se desarrolló el proyecto.

3.2 Clonar el repositorio.

Clonar significa descargar una copia completa del proyecto desde GitHub, incluyendo todo el historial de cambios:

```
git clone https://github.com/SalvaIES/payment-gateway-api.git  
cd payment-gateway-api
```

Después de ejecutar esto tendrás una carpeta `payment-gateway-api` con todo el código del proyecto. El comando `cd` entra dentro de esa carpeta.

3.3 Crear el entorno virtual.

Un entorno virtual es una carpeta aislada que tiene su propia instalación de Python y sus propias librerías. Sin entorno virtual, todas las librerías se instalan globalmente en el sistema y pueden entrar en conflicto entre proyectos.

Por ejemplo, imagina que tienes dos proyectos: uno necesita Django 4.2 y otro necesita Django 6.0. Sin entorno virtual no pueden coexistir. Con entorno virtual cada uno tiene el suyo.

En Linux con Python 3.12:

```
python3 -m venv venv --without-pip  
source venv/bin/activate  
curl https://bootstrap.pypa.io/get-pip.py | python
```

En Windows (PowerShell):

Primero hay que permitir ejecutar scripts (solo la primera vez):

```
Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Luego crear y activar el entorno:

```
python -m venv venv  
.\venv\Scripts\activate
```

Cuando el entorno virtual está activo, el prompt del terminal muestra **(venv)** al principio. Eso indica que cualquier librería que instales irá al entorno virtual, no al sistema global.

Importante: cada vez que abres un terminal nuevo hay que volver a activar el entorno virtual. El proyecto no funcionará sin él.

3.4 Instalar las librerías.

Con el entorno virtual activo, instala todas las librerías del proyecto con un solo comando:

```
pip install -r requirements.txt
```

pip es el gestor de paquetes de Python. El archivo **requirements.txt** le dice qué instalar:

```
Django>=5.2  
djangoRESTframework  
python-dotenv  
requests  
sqlparse  
stripe  
pyredsys  
paypalrestsdk  
flake8  
tzdata
```

El símbolo **>=** significa "esta versión o superior". Así el proyecto funciona tanto en Python 3.10 (que instala Django 5.2) como en Python 3.12 (que instala Django 6.0).

pip descarga cada librería de internet y la instala en el entorno virtual. Verás muchas líneas de texto mientras descarga e instala. Al final debe decir **Successfully installed**

3.5 Crear el archivo **.env**.

Este es uno de los pasos más importantes. El archivo **.env** contiene todas las claves secretas del proyecto. **Nunca se sube a GitHub** porque si alguien lo viera podría hacer pagos reales con tus cuentas.

Crea el archivo en la raíz del proyecto:

```
nano .env
```

Y escribe esto, sustituyendo los valores por los tuyos reales:

```
# Stripe
STRIPE_SECRET_KEY=sk_test_51TTIkACQLY7w...
STRIPE_PUBLISHABLE_KEY=pk_test_51TTIkACQLY7w...
STRIPE_WEBHOOK_SECRET=whsec_be7f582565...

# Redsys (estas son las credenciales públicas de prueba)
REDSYS_SECRET_KEY=sq7HjrUOBfKmc576ILgskD5srU870gJ7
REDSYS_MERCHANT_CODE=999008881
REDSYS_TERMINAL=1
REDSYS_URL=https://sis-t.redsys.es:25443/sis/realizarPago

# PayPal
PAYPAL_CLIENT_ID=AcXYVuNGqeQh12lWql8Y...
PAYPAL_CLIENT_SECRET=EHwRMP03xuhEyZpT6x...
PAYPAL_URL=https://api-m.sandbox.paypal.com
```

¿Cómo se leen estas variables en el código? En `config/settings.py`:

```
from dotenv import load_dotenv
import os

load_dotenv() # lee el archivo .env

STRIPE_SECRET_KEY = os.getenv('STRIPE_SECRET_KEY')
STRIPE_PUBLISHABLE_KEY = os.getenv('STRIPE_PUBLISHABLE_KEY')
STRIPE_WEBHOOK_SECRET = os.getenv('STRIPE_WEBHOOK_SECRET')
REDSYS_SECRET_KEY = os.getenv('REDSYS_SECRET_KEY')
REDSYS_MERCHANT_CODE = os.getenv('REDSYS_MERCHANT_CODE')
REDSYS_TERMINAL = os.getenv('REDSYS_TERMINAL')
REDSYS_URL = os.getenv('REDSYS_URL')
PAYPAL_CLIENT_ID = os.getenv('PAYPAL_CLIENT_ID')
PAYPAL_CLIENT_SECRET = os.getenv('PAYPAL_CLIENT_SECRET')
PAYPAL_URL = os.getenv('PAYPAL_URL')
```

`os.getenv('NOMBRE')` lee la variable de entorno con ese nombre. Si no existe devuelve **None**.

3.6 Configurar **settings.py**.

El archivo **config/settings.py** es la configuración central de Django. Las partes más importantes son las siguientes.

INSTALLED_APPS: le dice a Django qué aplicaciones están activas:

```
INSTALLED_APPS = [  
    'django.contrib.admin',      # panel de administración  
    'django.contrib.auth',      # sistema de usuarios  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  # sesiones de usuario  
    'django.contrib.messages',  # mensajes flash  
    'django.contrib.staticfiles', # archivos estáticos  
    'rest_framework',          # Django REST Framework  
    'rest_framework.authtoken', # autenticación por token  
    'core',                    # nuestra app principal  
    'frontend',                 # nuestro panel web  
]
```

REST_FRAMEWORK: configuración de DRF.

```
REST_FRAMEWORK = {  
    'DEFAULT_AUTHENTICATION_CLASSES': [  
        'rest_framework.authentication.TokenAuthentication',  
    ],  
    'DEFAULT_PERMISSION_CLASSES': [  
        'rest_framework.permissions.IsAuthenticated',  
    ],  
}
```

Esto le dice a DRF que por defecto todos los **endpoints** requieren autenticación por **token**. Si no tienes token válido, la API devuelve **error 401**.

DATABASES: configuración de la base de datos.

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.sqlite3',  
        'NAME': BASE_DIR / 'db.sqlite3',  
    }  
}
```

Usa SQLite y guarda el archivo `db.sqlite3` en la raíz del proyecto.

ALLOWED_HOSTS: qué dominios pueden acceder.

```
ALLOWED_HOSTS = ['127.0.0.1', 'localhost', '.ngrok-free.dev']
```

El `.ngrok-free.dev` permite el acceso desde cualquier subdominio de **ngrok**, necesario para las pruebas de **Redsys**.

CSRF_TRUSTED_ORIGINS: dominios de confianza para formularios.

```
CSRF_TRUSTED_ORIGINS = ['https://xxxx.ngrok-free.dev']
```

Django protege los formularios contra ataques **CSRF**. Esta configuración le dice que los formularios enviados desde **ngrok** son legítimos.

DEFAULT_AUTO_FIELD: tipo de clave primaria por defecto.

```
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
```

Esto evita un warning de **Django**. Define que los **IDs** de los modelos sean enteros grandes (**BigInteger**) en lugar de enteros normales.

3.7 Aplicar las migraciones.

Las migraciones son archivos que describen cómo debe ser la base de datos. Django genera estos archivos automáticamente a partir de los modelos y los aplica a la base de datos:

```
python manage.py makemigrations
python manage.py migrate
```

`makemigrations` detecta los cambios en los modelos y genera los archivos de migración en `core/migrations/`. `migrate` aplica esos archivos a la base de datos, creando las tablas.

Las migraciones del proyecto son:

- `0001_initial.py`: crea las tablas **Provider** y **Transaction**.
- `0002_transaction_stripe_payment_intent_id.py`: añade el campo **stripe_payment_intent_id**.
- `0003_transaction_stripe_client_secret.py`: añade el campo **stripe_client_secret**.

3.8 Crear el superusuario.

El superusuario es el administrador del sistema. Es necesario para poder hacer login tanto en el panel web como en la API:

```
python manage.py createsuperuser
```

Django preguntará el nombre de usuario, email (opcional) y contraseña. En desarrollo usamos credenciales sencillas como `admin` / `admin1234`.

3.9 Arrancar el servidor.

```
python manage.py runserver
```

Esto arranca el servidor de desarrollo de **Django** en <http://127.0.0.1:8000/>. Verás algo como:

```
Django version 6.0.4, using settings 'config.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.
```

El servidor se recarga automáticamente cada vez que guardas un archivo Python. No es necesario reiniciarlo manualmente al hacer cambios en el código.

3.10 Servicios adicionales necesarios.

Además del servidor **Django**, para probar los pagos necesitas dos servicios más en terminales separados:

Terminal 2: Stripe CLI (para que los webhooks de Stripe lleguen al servidor local).

```
stripe login
stripe listen --forward-to http://127.0.0.1:8000/api/webhooks/stripe/
```

Terminal 3: ngrok (solo para probar Redsys).

```
ngrok http 8000
```

ngrok te dará una URL pública como <https://outscore-footless-pebbly.ngrok-free.dev>. Esa URL hay que actualizarla en `core/views.py` en la función `redsys_payment` y en `config/settings.py` en `CSRF_TRUSTED_ORIGINS`.

4. La estructura del proyecto en detalle.

Vamos a recorrer cada carpeta y archivo explicando exactamente qué hace y por qué existe.

4.1 La raíz del proyecto.

```
payment-gateway-api/  
├─ config/  
├─ core/  
├─ frontend/  
├─ docs/  
├─ backlog/  
├─ manage.py  
├─ requirements.txt  
├─ .env  
├─ .gitignore  
├─ LICENSE  
└─ README.md
```

manage.py: es la navaja suiza de Django. Es un script que acepta comandos para gestionar el proyecto:

```
python manage.py runserver      # arranca el servidor  
python manage.py migrate       # aplica migraciones  
python manage.py createsuperuser # crea un administrador  
python manage.py test          # ejecuta los tests  
python manage.py shell         # abre una consola Python con Django cargado
```

Django lo genera automáticamente cuando creas un proyecto con **django-admin startproject**.

.gitignore: le dice a Git qué archivos ignorar y nunca subir a GitHub. Los más importantes son:

```
.env          # claves secretas  
db.sqlite3    # base de datos local  
venv/         # entorno virtual  
__pycache__/  # archivos compilados de Python
```

README.md: descripción del proyecto en formato Markdown. Es lo primero que ve cualquiera que visita el repositorio en GitHub.

4.2 La carpeta **config/**.

```
config/  
├─ __init__.py  
├─ settings.py  
├─ urls.py  
├─ asgi.py  
└─ wsgi.py
```

Esta carpeta contiene la configuración global de Django. Se creó con el comando:

```
django-admin startproject config .
```

El punto al final es importante, hace que **manage.py** quede en la raíz en lugar de en una subcarpeta.

settings.py: ya lo explicamos en la sección anterior. Es el cerebro de la configuración.

urls.py: es el enrutador principal. Define qué vista gestiona cada URL:

```
from django.contrib import admin  
from django.urls import path, include  
from rest_framework.auth_token.views import obtain_auth_token  
from core.views import (  
    stripe_webhook, redsys_payment, redsys_webhook,  
    stripe_payment, paypal_payment, paypal_execute  
)  
  
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('api/', include('core.urls')),  
    path('api/auth/login/', obtain_auth_token, name='api_token_auth'),  
    path('api/webhooks/stripe/', stripe_webhook, name='stripe_webhook'),  
    path('api/webhooks/redsyst/', redsys_webhook, name='redsyst_webhook'),  
    path('api/pay/redsyst/<int:transaction_id>/', redsys_payment, name='redsyst_payment'),  
    path('api/pay/stripe/<int:transaction_id>/', stripe_payment, name='stripe_payment'),  
    path('api/pay/paypal/<int:transaction_id>/', paypal_payment, name='paypal_payment'),  
    path('api/pay/paypal/execute/', paypal_execute, name='paypal_execute'),  
    path("", include('frontend.urls')),  
]
```

Cada `path()` define una regla: cuando la URL coincide con el patrón, Django llama a la vista correspondiente.

- `path('admin/', ...)`: el panel de administración de Django.
- `path('api/', include('core.urls'))`: delega todas las URLs que empiezan por `/api/` a `core/urls.py`.
- `path('api/auth/login/', obtain_auth_token)`: endpoint de login que devuelve el token.
- `path('api/webhooks/stripe/', stripe_webhook)`: recibe notificaciones de Stripe.
- `path('api/pay/redsys/<int:transaction_id>/', redsys_payment)`: inicia el pago con Redsys. El `<int:transaction_id>` captura el número de la transacción de la URL.
- `path('', include('frontend.urls'))`: delega el resto de URLs al frontend.

asgi.py y wsgi.py: son interfaces para desplegar Django en servidores de producción. En desarrollo no se usan.

__init__.py: archivo vacío que le dice a Python que esta carpeta es un módulo Python. Todos los paquetes Python lo tienen.

4.3 La carpeta **core/**.

```
core/
├── __init__.py
├── admin.py
├── apps.py
├── models.py
├── serializers.py
├── views.py
├── urls.py
├── tests.py
├── migrations/
│   ├── 0001_initial.py
│   ├── 0002_transaction_stripe_payment_intent_id.py
│   └── 0003_transaction_stripe_client_secret.py
└── templates/
    ├── core/
    │   ├── redsys_payment.html
    │   └── stripe_payment.html
```

Esta es la aplicación principal. Se creó con:

```
python manage.py startapp core
```

admin.py: registra los modelos en el panel de administración de Django:

```
from django.contrib import admin
from .models import Provider, Transaction

admin.site.register(Provider)
admin.site.register(Transaction)
```

Gracias a esto, en <http://127.0.0.1:8000/admin/> puedes ver, crear, editar y eliminar proveedores y transacciones directamente desde el navegador, sin escribir código.

apps.py: configuración de la aplicación. Django lo genera automáticamente:

```
from django.apps import AppConfig

class CoreConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'core'
```

migrations/: carpeta con los archivos de migración. Cada vez que cambias un modelo y ejecutas `makemigrations`, Django genera un nuevo archivo aquí. Estos archivos describen exactamente qué cambios hay que hacer en la base de datos.

templates/core/: plantillas HTML específicas de la app core. Contiene los formularios de pago de Stripe y Redsys.

4.4 La carpeta **frontend/**.

```
frontend/  
├── __init__.py  
├── admin.py  
├── apps.py  
├── models.py  
├── views.py  
├── urls.py  
├── tests.py  
└── templates/  
    └── frontend/  
        ├── base.html  
        ├── login.html  
        ├── dashboard.html  
        ├── providers.html  
        └── transactions.html
```

Esta aplicación gestiona el panel web visual. Se creó con:

```
python manage.py startapp frontend
```

views.py: contiene las funciones que generan las páginas HTML. Cada función recibe una petición HTTP y devuelve una respuesta HTML.

urls.py: define las rutas del frontend:

```
from django.urls import path  
from . import views  
  
urlpatterns = [  
    path("", views.dashboard, name='dashboard'),  
    path("login/", views.login_view, name='login'),  
    path("logout/", views.logout_view, name='logout'),  
    path("providers/", views.providers, name='providers'),  
    path("providers/<int:provider_id>/delete/", views.delete_provider, name='delete_provider'),  
    path("transactions/", views.transactions, name='transactions'),  
    path("transactions/<int:transaction_id>/update/", views.update_transaction, name='update_transaction'),  
    path("transactions/<int:transaction_id>/delete/", views.delete_transaction, name='delete_transaction'),  
]
```

templates/frontend/: los archivos HTML del panel. Django los busca automáticamente dentro de las carpetas **templates/** de cada app porque en **settings.py** está configurado **APP_DIRS: True**.

4.5 La carpeta **docs/**.

```
docs/  
├─ memoria_tecnica.md  
└─ manual_usuario.md
```

Contiene la documentación del proyecto en formato Markdown. Son documentos de texto con formato que GitHub renderiza automáticamente.

4.6 La carpeta **backlog/**.

```
backlog/  
└─ product_backlog.md
```

Contiene el Product Backlog del proyecto siguiendo metodología SCRUM. Lista todas las User Stories organizadas por épicas y sprints.

4.7 Los archivos de migración en detalle.

Las migraciones merecen una explicación especial porque son fundamentales para entender cómo Django gestiona la base de datos.

0001_initial.py: crea las tablas iniciales:

```
class Migration(migrations.Migration):
    operations = [
        migrations.CreateModel(
            name='Provider',
            fields=[
                ('id', models.BigAutoField(...)),
                ('name', models.CharField(max_length=100)),
                ('api_key', models.CharField(max_length=255)),
                ('environment', models.CharField(...)),
                ('is_active', models.BooleanField(default=True)),
                ('created_at', models.DateTimeField(auto_now_add=True)),
            ],
        ),
        migrations.CreateModel(
            name='Transaction',
            fields=[...],
        ),
    ]
```

0002_transaction_stripe_payment_intent_id.py: añade el campo cuando integramos Stripe:

```
class Migration(migrations.Migration):
    operations = [
        migrations.AddField(
            model_name='transaction',
            name='stripe_payment_intent_id',
            field=models.CharField(blank=True, default="", max_length=255),
        ),
    ]
```

0003_transaction_stripe_client_secret.py: añade el campo para el formulario de pago de Stripe:

```
class Migration(migrations.Migration):
    operations = [
        migrations.AddField(
            model_name='transaction',
            name='stripe_client_secret',
            field=models.CharField(blank=True, default="", max_length=255),
        ),
    ]
```

Cada migración tiene una dependencia con la anterior, formando una cadena. Django las aplica en orden y recuerda cuáles ya ha aplicado para no repetirlas.

5. Los modelos. La estructura de la base de datos.

Los modelos son el corazón de cualquier aplicación Django. Definen cómo se guardan los datos y Django los traduce automáticamente a tablas de base de datos. Están en `core/models.py`.

5.1 ¿Qué es un modelo Django?

Un modelo es una clase Python que hereda de `models.Model`. Cada atributo de la clase se convierte en una columna de la tabla. Django se encarga de:

- Crear la tabla en la base de datos
- Generar el SQL necesario para insertar, consultar, actualizar y eliminar
- Validar los tipos de datos
- Gestionar las relaciones entre tablas

No necesitas escribir SQL. Django lo hace todo por ti.

5.2 El modelo Provider (Proveedor).

```
class Provider(models.Model):
    """Representa una pasarela de pago (Stripe, PayPal, Redsys)."""

    ENVIRONMENT_CHOICES = [
        ('sandbox', 'Sandbox'),
        ('production', 'Production'),
    ]

    name = models.CharField(max_length=100)
    api_key = models.CharField(max_length=255)
    environment = models.CharField(
        max_length=20,
        choices=ENVIRONMENT_CHOICES,
        default='sandbox'
    )
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        """Devuelve la representación en texto del proveedor."""
        return f"{self.name} ({self.environment})"

    class Meta:
        ordering = ['name']
```

Vamos línea a línea:

ENVIRONMENT_CHOICES: es una lista de tuplas que define los valores permitidos para el campo `environment`. Cada tupla tiene dos elementos: el valor que se guarda en la base de datos y el texto que se muestra al usuario. Con esto Django valida automáticamente que solo se puedan usar esos valores.

name = models.CharField(max_length=100): campo de texto de máximo 100 caracteres. Guarda el nombre del proveedor: "Stripe", "PayPal", "Redsys".

api_key = models.CharField(max_length=255): campo de texto para la clave de API. 255 caracteres porque las claves de las pasarelas son largas.

environment = models.CharField(max_length = 20, choices = ENVIRONMENT_CHOICES, default = 'sandbox'): campo con opciones limitadas. `default='sandbox'` significa que si no se especifica, se asume sandbox (entorno de pruebas).

is_active = models.BooleanField(default=True): campo booleano (verdadero/falso). Permite desactivar un proveedor sin eliminarlo, manteniendo el historial de transacciones.

created_at = models.DateTimeField(auto_now_add=True): fecha y hora de creación. `auto_now_add=True` hace que Django rellene este campo automáticamente con la fecha actual cuando se crea el objeto. No se puede modificar después.

def __str__(self): define cómo se muestra el objeto como texto. Cuando Django necesita mostrar un proveedor como cadena (por ejemplo en el panel de administración), llama a este método. Devuelve algo como "Stripe (sandbox)".

class Meta: configuración adicional del modelo. **ordering = ['name']** hace que por defecto los proveedores se devuelvan ordenados alfabéticamente por nombre.

5.3 El modelo Transaction (Transacción).

```
class Transaction(models.Model):
    """Registra cada intento de pago asociado a un proveedor."""

    STATUS_CHOICES = [
        ('pending', 'Pending'),
        ('completed', 'Completed'),
        ('failed', 'Failed'),
    ]

    INCIDENT_CHOICES = [
        ('', 'Sin incidencia'),
        ('impago', 'Impago'),
        ('error_conexion', 'Error de conexión'),
        ('devolucion', 'Devolución'),
    ]

    provider = models.ForeignKey(
        Provider,
        on_delete=models.PROTECT,
        related_name='transactions'
    )
    amount = models.DecimalField(max_digits=10, decimal_places=2)
    currency = models.CharField(max_length=3, default='EUR')
    status = models.CharField(
        max_length=20,
        choices=STATUS_CHOICES,
        default='pending'
    )
    incident_type = models.CharField(
        max_length=20,
        choices=INCIDENT_CHOICES,
        blank=True,
        default=""
    )
    stripe_payment_intent_id = models.CharField(
        max_length=255,
        blank=True,
        default=""
    )
    stripe_client_secret = models.CharField(
        max_length=255,
        blank=True,
        default=""
    )
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    def __str__(self):
        """Devuelve la representación en texto de la transacción."""
        return (
            f"{self.provider.name}: {self.amount} "
            f"{self.currency} ({self.status})"
        )

    class Meta:
        ordering = ['-created_at']
```

Línea a línea:

STATUS_CHOICES: los tres estados posibles de una transacción:

- **pending**: el pago se ha iniciado pero no se ha completado aún.
- **completed**: el pago se ha procesado con éxito.
- **failed**: el pago ha fallado.

INCIDENT_CHOICES: los tipos de incidencia posibles. La primera tupla `('', 'Sin incidencia')` permite que el campo esté vacío cuando no hay incidencia.

provider = models.ForeignKey(Provider, on_delete=models.PROTECT, related_name='transactions'): este es el campo más importante del modelo. Establece la relación con el modelo Provider.

- **ForeignKey** significa que cada transacción está vinculada a exactamente un proveedor.
- **on_delete=models.PROTECT** significa que si intentas eliminar un proveedor que tiene transacciones, Django lanzará un error y no lo permitirá. Esto protege la integridad histórica de los datos: no puedes borrar un proveedor si tiene pagos registrados.
- **related_name='transactions'** permite acceder a todas las transacciones de un proveedor con `proveedor.transactions.all()`.

amount = models.DecimalField(max_digits=10, decimal_places=2): campo numérico decimal para el importe. `max_digits=10` significa máximo 10 dígitos en total y `decimal_places=2` significa 2 decimales. Permite importes como 99999999.99 (casi cien millones). Se usa Decimal en lugar de Float porque los flotantes tienen problemas de precisión con dinero.

currency = models.CharField(max_length=3, default='EUR'): código ISO de moneda de 3 letras: EUR, USD, GBP..

incident_type = models.CharField(blank=True, default=''): `blank=True` permite que el campo esté vacío. Por defecto está vacío porque la mayoría de transacciones no tienen incidencia.

stripe_payment_intent_id: guarda el ID del PaymentIntent de Stripe. Ejemplo: `pi_3TVaYZCQLY7whPlj0rml7N66`. Se usa para relacionar nuestra transacción con el pago en Stripe y para procesar los webhooks.

stripe_client_secret: guarda el client secret del PaymentIntent. Ejemplo: `pi_3TVaYZCQLY7whPlj0rml7N66_secret_wFYeas...`. Es necesario para que el formulario de pago de Stripe pueda confirmar el pago desde el navegador del usuario.

updated_at = models.DateTimeField(auto_now=True): a diferencia de `auto_now_add`, `auto_now=True` actualiza la fecha cada vez que se guarda el objeto, no solo al crearlo. Así sabemos cuándo se modificó por última vez.

ordering = ['-created_at']: el `-` antes del nombre del campo significa orden descendente. Las transacciones más recientes aparecen primero.

5.4 La relación entre los modelos.

La relación es **uno a muchos (1:N)**:

```
Provider (1) ↔ Transaction (N)
```

Un proveedor puede tener muchas transacciones, pero cada transacción pertenece a exactamente un proveedor.

En la base de datos esto se implementa con una **clave foránea** (foreign key): la tabla **Transaction** tiene una columna **provider_id** que guarda el ID del proveedor al que pertenece.

Django gestiona esto automáticamente. Cuando haces **transaction.provider** Django hace una consulta SQL internamente para traer el proveedor correspondiente.

5.5 El ORM de Django: consultas sin SQL.

Django tiene un sistema llamado ORM (Object Relational Mapper) que permite hacer consultas a la base de datos usando Python en lugar de SQL. Algunos ejemplos usados en el proyecto:

Obtener todas las transacciones:

```
Transaction.objects.all()
# SQL: SELECT * FROM core_transaction
```

Filtrar por estado:

```
Transaction.objects.filter(status='pending')
# SQL: SELECT * FROM core_transaction WHERE status = 'pending'
```

Filtrar por proveedor y fecha:

```
Transaction.objects.filter(
    provider__id=1,
    created_at__date='2026-05-10'
)
# SQL: SELECT * FROM core_transaction
# WHERE provider_id = 1
# AND DATE(created_at) = '2026-05-10'
```

El doble guión bajo **__** es la sintaxis de Django para acceder a campos relacionados y aplicar filtros especiales. **provider__id** significa "el campo id del proveedor relacionado". **created_at__date** significa "la parte de fecha del campo created_at".

Actualizar el estado de una transacción:

```
Transaction.objects.filter(
    stripe_payment_intent_id='pi_3TVaYZ...'
).update(status='completed')
# SQL: UPDATE core_transaction
#   SET status = 'completed'
#   WHERE stripe_payment_intent_id = 'pi_3TVaYZ...'
```

Esta forma de actualizar es especialmente eficiente porque hace una sola consulta SQL sin cargar el objeto en memoria.

Obtener un objeto por ID:

```
transaction = Transaction.objects.get(id=1)
# SQL: SELECT * FROM core_transaction WHERE id = 1
```

Si no existe lanza `Transaction.DoesNotExist`. Por eso en el código siempre se usa dentro de un try/except:

```
try:
    transaction = Transaction.objects.get(id=transaction_id)
except Transaction.DoesNotExist:
    return HttpResponse(status=404)
```

6. Los serializers. Validación y conversión de datos.

Los serializers están en `core/serializers.py`. Son uno de los componentes más importantes de Django REST Framework y merece una explicación detallada.

6.1 ¿Qué es un serializer y para qué sirve?

Un serializer hace dos trabajos opuestos según la dirección del flujo de datos:

De Python a JSON (serialización), cuando la API devuelve datos al cliente:

```
Objeto Python (Transaction) → Serializer → JSON
{"id": 1, "amount": "99.99", "status": "pending"}
```

De JSON a Python (deserialización), cuando el cliente envía datos a la API:

```
JSON → Serializer → Objeto Python validado → Base de datos
{"provider": 1, "amount": "99.99", "currency": "EUR"}
```

En la deserialización el serializer también **valida** los datos: comprueba que los tipos son correctos, que los campos obligatorios están presentes y aplica las reglas de negocio que definamos.

Sin serializers tendríamos que hacer toda esta conversión y validación manualmente, lo que sería muy tedioso y propenso a errores.

6.2 El ProviderSerializer.

```
class ProviderSerializer(serializers.ModelSerializer):
    """Serializer para el modelo Provider."""

    class Meta:
        model = Provider
        fields = '__all__'
        read_only_fields = ['created_at']
```

serializers.ModelSerializer: hereda de `ModelSerializer`, que es la clase base de DRF para serializers vinculados a modelos. Genera automáticamente los campos a partir del modelo.

class Meta: configuración del serializer:

- **model = Provider**: le dice qué modelo usar
- **fields = '__all__'**: incluye todos los campos del modelo en el JSON
- **read_only_fields = ['created_at']**: el campo **created_at** es de solo lectura: se devuelve en las respuestas pero no se puede enviar en las peticiones. Tiene sentido porque Django lo rellena automáticamente.

Este serializer es sencillo porque `Provider` no necesita validaciones especiales. Cualquier nombre, clave API y entorno válido se acepta.

6.3 El TransactionSerializer.

Este es más complejo porque necesita validaciones de negocio:

```
class TransactionSerializer(serializers.ModelSerializer):
    """Serializer para el modelo Transaction."""

    provider_name = serializers.CharField(
        source='provider.name',
        read_only=True
    )

    class Meta:
        model = Transaction
        fields = '__all__'
        read_only_fields = [
            'created_at',
            'updated_at',
            'stripe_client_secret'
        ]
```

provider_name = serializers.CharField(source='provider.name', read_only=True): este es un campo extra que no existe en el modelo pero que añadimos al JSON de respuesta.

¿Por qué? Porque cuando el frontend recibe una transacción, el campo **provider** solo devuelve el ID numérico del proveedor (ej. **1**). Para mostrar el nombre ("Stripe") en la tabla tendría que hacer otra petición a la API para buscar el proveedor por ID. Eso son dos peticiones innecesarias.

Con **provider_name** incluimos directamente el nombre en la respuesta:

```
{
  "id": 1,
  "provider": 1,
  "provider_name": "Stripe",
  "amount": "99.99",
  "status": "pending"
}
```

source='provider.name' le dice al serializer de dónde sacar el dato: accede al proveedor relacionado y toma su campo **name**. **read_only=True** significa que no se puede enviar en las peticiones, solo se devuelve.

read_only_fields incluye tres campos:

- **created_at**: fecha de creación automática
- **updated_at**: fecha de modificación automática
- **stripe_client_secret**: este es especialmente importante: el client secret es sensible y nunca debe llegar desde el exterior. Solo el propio backend lo puede establecer cuando crea el PaymentIntent en Stripe.

6.4 La validación del importe.

```
def validate_amount(self, value):
    """El importe debe ser mayor que cero."""
    if value <= 0:
        raise serializers.ValidationError(
            "El importe debe ser mayor que cero."
        )
    return value
```

Los métodos que empiezan por `validate_` seguido del nombre de un campo son validadores de campo. Django REST Framework los llama automáticamente cuando deserializa datos.

Este método recibe el valor del campo `amount` ya convertido al tipo correcto (Decimal) y comprueba que sea positivo. Si no lo es, lanza una excepción `ValidationError` que DRF convierte automáticamente en una respuesta HTTP 400 con el mensaje de error:

```
{
  "amount": ["El importe debe ser mayor que cero."]
}
```

Si el valor es válido, lo devuelve tal cual con `return value`.

6.5 La validación cruzada de campos.

```
def validate(self, data):
    """Si el estado es failed, el tipo de incidencia es obligatorio."""
    if data.get('status') == 'failed' and not data.get('incident_type'):
        raise serializers.ValidationError(
            "Debe especificar un tipo de incidencia "
            "cuando el estado es 'failed'."
        )
    return data
```

El método **validate** (sin nombre de campo) es un validador global que recibe todos los datos a la vez. Sirve para validaciones que dependen de múltiples campos simultáneamente.

Esta validación implementa una regla de negocio: **si una transacción falla, es obligatorio indicar por qué falló**. No tiene sentido marcar una transacción como fallida sin saber si fue un impago, un error de conexión o una devolución.

data.get('status') obtiene el valor del campo status del diccionario de datos (devuelve None si no existe, en lugar de lanzar un error). **not data.get('incident_type')** comprueba que el tipo de incidencia esté vacío.

Si la condición se cumple (status es failed Y incident_type está vacío), lanza el error. La respuesta sería:

```
{
  "non_field_errors": [
    "Debe especificar un tipo de incidencia cuando el estado es 'failed'."
  ]
}
```

6.6 Cómo funciona la serialización en la práctica.

Cuando el cliente hace una petición GET para ver las transacciones:

1. La vista obtiene los objetos de la base de datos: **Transaction.objects.all()**
2. Pasa los objetos al serializer: **TransactionSerializer(queryset, many=True)**
3. El serializer convierte cada objeto a un diccionario Python.
4. DRF convierte los diccionarios a JSON.
5. La API devuelve el JSON al cliente.

Cuando el cliente hace una petición POST para crear una transacción:

1. DRF recibe el JSON del cuerpo de la petición.
2. Lo pasa al serializer: **TransactionSerializer(data=request.data)**
3. El serializer llama a **is_valid()** que ejecuta todas las validaciones.
4. Si hay errores devuelve HTTP 400 con los mensajes.
5. Si todo está bien, llama a **save()** que crea el objeto en la base de datos.
6. Serializa el objeto creado y lo devuelve como JSON con HTTP 201.

6.7 Por qué los importes son strings en el JSON.

Si miras las respuestas de la API verás que el importe llega como string:

```
{"amount": "99.99"}
```

No como número:

```
{"amount": 99.99}
```

Esto es deliberado. Los números decimales en JSON son flotantes, y los flotantes tienen problemas de precisión con dinero. Por ejemplo, $0.1 + 0.2$ en JavaScript da 0.30000000000000004 . Django usa el tipo `Decimal` de Python para evitar esto y lo serializa como string para preservar la precisión exacta.

7. Las vistas de la API. La lógica principal.

Las vistas están en `core/views.py`. Es el archivo más largo y complejo del proyecto. Aquí está toda la lógica: los endpoints de la API, la integración con las tres pasarelas de pago y los webhooks.

7.1 Los imports, qué librerías se usan y por qué.

```
from datetime import datetime
import stripe
from django.conf import settings
from django.views.decorators.csrf import csrf_exempt
from django.http import HttpResponse
from django.shortcuts import render
from rest_framework import viewsets, filters, status
from rest_framework.authentication import TokenAuthentication
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from pyresys.payment import request_payment, MerchantParameters
from pyresys.notification import validate_notification, SignatureVerificationError
import paypalrestsdk
from .models import Provider, Transaction
from .serializers import ProviderSerializer, TransactionSerializer
```

Línea a línea:

- `from datetime import datetime`: para generar números de pedido únicos basados en la fecha y hora.
- `import stripe`: el SDK oficial de Stripe.
- `from django.conf import settings`: para acceder a las variables de `settings.py` (claves de API, etc.).
- `from django.views.decorators.csrf import csrf_exempt`: para desactivar la protección CSRF en los webhooks. Los webhooks vienen de servidores externos (Stripe, Redsys) que no tienen token CSRF.
- `from django.http import HttpResponse`: para devolver respuestas HTTP simples.
- `from django.shortcuts import render`: para renderizar plantillas HTML.
- `from rest_framework import viewsets, filters, status`: ViewSets para la API, filtros de búsqueda y códigos de estado HTTP.
- `from rest_framework.authentication import TokenAuthentication`: autenticación por token.
- `from rest_framework.permissions import IsAuthenticated`: permiso que requiere autenticación.
- `from rest_framework.response import Response`: respuestas JSON de DRF.

- `from pyredsys.payment import request_payment, MerchantParameters`: para generar el formulario firmado de Redsys.
- `from pyredsys.notification import validate_notification, SignatureVerificationError`: para verificar las notificaciones de Redsys.
- `import paypalrestsdk`: el SDK de PayPal.
- `from .models import Provider, Transaction`: los modelos del proyecto.
- `from .serializers import ProviderSerializer, TransactionSerializer`: los serializers.

Al final de los imports se configura Stripe globalmente:

```
stripe.api_key = settings.STRIPE_SECRET_KEY
```

Esto establece la clave secreta de Stripe para toda la aplicación. Todas las llamadas al SDK de Stripe usarán esta clave automáticamente.

7.2 El **ProviderViewSet**.

```
class ProviderViewSet(viewsets.ModelViewSet):
    """CRUD completo para proveedores de pago."""

    queryset = Provider.objects.all()
    serializer_class = ProviderSerializer
    authentication_classes = [TokenAuthentication]
    permission_classes = [IsAuthenticated]
    filter_backends = [filters.SearchFilter]
    search_fields = ['name', 'environment']
```

viewsets.ModelViewSet: esta clase base de DRF genera automáticamente todos los endpoints CRUD con muy poco código:

Método HTTP	URL	Acción	Descripción
GET	/api/providers/	list	Lista todos los proveedores
POST	/api/providers/	create	Crea un proveedor
GET	/api/providers/1/	retrieve	Devuelve el proveedor con ID 1
PUT	/api/providers/1/	update	Actualiza el proveedor con ID 1
DELETE	/api/providers/1/	destroy	Elimina el proveedor con ID 1

Sin DRF tendríamos que escribir una función para cada una de estas acciones. Con ModelViewSet son unas pocas líneas.

queryset = Provider.objects.all(): define qué objetos gestiona este ViewSet. **Provider.objects.all()** devuelve todos los proveedores de la base de datos.

serializer_class = ProviderSerializer: qué serializer usar para convertir los datos.

authentication_classes = [TokenAuthentication]: define cómo se autentica el usuario. Con TokenAuthentication el cliente debe incluir el token en la cabecera de cada petición:

```
Authorization: Token a7879add4e69b2310b699f48a37ee90e11e5dc27
```

permission_classes = [IsAuthenticated]: define quién puede acceder. **IsAuthenticated** significa que solo los usuarios autenticados (con token válido) pueden usar este endpoint. Si no hay token o es inválido, DRF devuelve automáticamente HTTP 401.

filter_backends = [filters.SearchFilter]: activa el sistema de búsqueda. Permite filtrar con **?search=stripe**.

search_fields = ['name', 'environment']: qué campos se buscan. Con **?search=stripe** busca en el nombre y el entorno.

7.3 El **TransactionViewSet**, la parte básica.

```
class TransactionViewSet(viewsets.ModelViewSet):
    """CRUD completo para transacciones con filtros."""

    queryset = Transaction.objects.all()
    serializer_class = TransactionSerializer
    authentication_classes = [TokenAuthentication]
    permission_classes = [IsAuthenticated]
    filter_backends = [filters.SearchFilter, filters.OrderingFilter]
    search_fields = ['status', 'incident_type', 'currency']
    ordering_fields = ['created_at', 'amount']
```

Similar al ProviderViewSet pero con dos filtros adicionales:

filters.OrderingFilter: permite ordenar los resultados. Con **?ordering=amount** ordena por importe ascendente. Con **?ordering=-amount** ordena descendente (el **-** indica descendente).

ordering_fields = ['created_at', 'amount']: solo se puede ordenar por estos campos. Si intentas **?ordering=api_key** DRF lo ignora por seguridad.

7.4 El método `get_queryset`. Filtros personalizados.

```
def get_queryset(self):
    """Filtra las transacciones según los parámetros de la URL."""
    queryset = Transaction.objects.all()
    provider = self.request.query_params.get('provider')
    status_param = self.request.query_params.get('status')
    date = self.request.query_params.get('date')

    if provider:
        queryset = queryset.filter(provider__id=provider)
    if status_param:
        queryset = queryset.filter(status=status_param)
    if date:
        queryset = queryset.filter(created_at__date=date)

    return queryset
```

Este método sobrescribe el queryset por defecto para añadir filtros personalizados.

`self.request.query_params.get('provider')`: lee el parámetro `provider` de la URL. Por ejemplo si la URL es `/api/transactions/?provider=1&status=pending`, `query_params.get('provider')` devuelve `'1'` y `query_params.get('status')` devuelve `'pending'`. Si el parámetro no está en la URL devuelve `None`.

`queryset = queryset.filter(provider__id=provider)`: aplica el filtro solo si el parámetro existe (el `if provider`: lo comprueba). El doble guión bajo `provider__id` es la sintaxis de Django para filtrar por el campo `id` del modelo relacionado `provider`.

`queryset.filter(created_at__date=date)`: `__date` extrae solo la parte de fecha de un campo `datetime`. Así `?date=2026-05-10` devuelve todas las transacciones del día 10 de mayo de 2026 independientemente de la hora.

Los filtros se encadenan: si pasas varios parámetros, se aplican todos:

```
/api/transactions/?provider=1&status=pending&date=2026-05-10
```

Devuelve solo las transacciones del proveedor 1, en estado pending, creadas el 10 de mayo.

7.5 El método **create**. Integración con Stripe al crear transacciones.

Este es el método más importante del proyecto. Sobreescribe el comportamiento por defecto de **create** para que al crear una transacción Stripe, automáticamente se cree también un `PaymentIntent` en Stripe:

```
def create(self, request, *args, **kwargs):
    """Crea la transacción y un PaymentIntent en Stripe si el proveedor es Stripe."""
    serializer = self.get_serializer(data=request.data)
    serializer.is_valid(raise_exception=True)

    provider = serializer.validated_data['provider']

    if provider.name.lower() == 'stripe':
        try:
            amount = serializer.validated_data['amount']
            currency = serializer.validated_data['currency'].lower()
            intent = stripe.PaymentIntent.create(
                amount=int(amount * 100),
                currency=currency,
                metadata={'provider': provider.name},
                automatic_payment_methods={
                    'enabled': True,
                    'allow_redirects': 'never'
                }
            )
            transaction = serializer.save(
                stripe_payment_intent_id=intent.id,
                stripe_client_secret=intent.client_secret
            )
        except stripe.StripeError as e:
            return Response(
                {'error': str(e)},
                status=status.HTTP_400_BAD_REQUEST
            )
        else:
            transaction = serializer.save()

    return Response(
        self.get_serializer(transaction).data,
        status=status.HTTP_201_CREATED
    )
```

Paso a paso:

`serializer = self.get_serializer(data=request.data)`: crea el serializer con los datos que envió el cliente en el cuerpo de la petición.

`serializer.is_valid(raise_exception=True)`: ejecuta todas las validaciones. Si algo falla, **`raise_exception=True`** hace que DRF devuelva automáticamente HTTP 400 con los errores, sin continuar la ejecución.

`provider = serializer.validated_data['provider']`: obtiene el objeto Provider ya validado. DRF ha verificado que ese ID de proveedor existe en la base de datos.

`if provider.name.lower() == 'stripe':`: comprueba si el proveedor es Stripe. **`.lower()`** convierte a minúsculas para que funcione tanto "Stripe" como "stripe" o "STRIPE".

`amount=int(amount * 100)`: Stripe trabaja en centimos, no en euros. 99.99€ se envía como 9999 céntimos. Multiplicamos por 100 y convertimos a entero.

`currency=currency.lower()`: Stripe requiere el código de moneda en minúsculas: "eur", no "EUR".

`metadata={'provider': provider.name}`: metadatos adicionales que se guardan en Stripe junto al pago. Útil para identificar el origen del pago en el dashboard de Stripe.

`automatic_payment_methods={'enabled': True, 'allow_redirects': 'never'}`: configura qué métodos de pago acepta. **`allow_redirects: 'never'`** es importante: sin esto Stripe intenta usar métodos como Bizum que requieren redirigir al usuario a otra página, lo que complica el flujo. Con esta opción solo acepta métodos que no requieren redirección (tarjeta de crédito).

`transaction = serializer.save(stripe_payment_intent_id=intent.id, stripe_client_secret=intent.client_secret)`: guarda la transacción en la base de datos pasando los datos de Stripe. **`serializer.save()`** acepta kwargs adicionales que se añaden al objeto antes de guardarlo.

`except stripe.StripeError as e:`: captura cualquier error de Stripe (tarjeta rechazada, credenciales incorrectas, etc.) y devuelve HTTP 400 con el mensaje de error.

Para otros proveedores: si el proveedor no es Stripe, simplemente se guarda la transacción sin llamar a ninguna API externa.

`status=status.HTTP_201_CREATED`: devuelve HTTP 201 (Created) en lugar de 200 (OK). La convención REST es usar 201 cuando se crea un recurso nuevo.

7.6 El **webhook** de Stripe.

```
@csrf_exempt
def stripe_webhook(request):
    """Recibe y procesa los eventos webhook de Stripe."""
    payload = request.body
    sig_header = request.META.get('HTTP_STRIPE_SIGNATURE')

    try:
        event = stripe.Webhook.construct_event(
            payload, sig_header, settings.STRIPE_WEBHOOK_SECRET
        )
    except ValueError:
        return HttpResponse(status=400)
    except stripe.errors.SignatureVerificationError:
        return HttpResponse(status=400)

    if event['type'] == 'payment_intent.succeeded':
        intent = event['data']['object']
        payment_intent_id = intent['id']
        Transaction.objects.filter(
            stripe_payment_intent_id=payment_intent_id
        ).update(status='completed')

    elif event['type'] == 'payment_intent.payment_failed':
        intent = event['data']['object']
        payment_intent_id = intent['id']
        Transaction.objects.filter(
            stripe_payment_intent_id=payment_intent_id
        ).update(status='failed', incident_type='impago')

    return HttpResponse(status=200)
```

Un webhook es una notificación que Stripe envía a nuestra API cuando ocurre algo. En lugar de que nosotros preguntemos constantemente "¿se ha pagado ya?", Stripe nos avisa cuando el pago se completa.

@csrf_exempt: desactiva la protección CSRF para esta vista. Normalmente Django exige un token CSRF en los formularios POST para evitar ataques. Pero Stripe no conoce nuestro token CSRF: es un servidor externo. Por eso lo desactivamos solo para el webhook.

payload = request.body: el cuerpo de la petición que envía Stripe. Contiene el evento en formato JSON.

sig_header = request.META.get('HTTP_STRIPE_SIGNATURE'): la firma digital que Stripe incluye en la cabecera de cada webhook. `request.META` contiene todas las cabeceras HTTP de la petición.

stripe.Webhook.construct_event(payload, sig_header, settings.STRIPE_WEBHOOK_SECRET): esta es la verificación de seguridad más importante. Stripe firma cada webhook con el **STRIPE_WEBHOOK_SECRET**. Este método verifica que la firma es válida, es decir, que el webhook realmente viene de Stripe y no de un atacante que intenta engañarnos para marcar pagos como completados.

Si la verificación falla lanza una excepción y devolvemos HTTP 400. Si alguien intenta enviar un webhook falso, lo rechazamos.

event['type']: el tipo de evento. Stripe envía muchos tipos de eventos (pagos creados, actualizados, fallidos...). Nos interesan dos:

- **payment_intent.succeeded**: el pago se completó con éxito → actualizamos a **completed**.
- **payment_intent.payment_failed**: el pago falló → actualizamos a **failed** con incidencia **impago**.

Transaction.objects.filter(...).update(...): actualiza directamente en la base de datos sin cargar el objeto en memoria. Es eficiente y atómico.

return HttpResponse(status=200): devolvemos 200 para confirmar a Stripe que recibimos el webhook. Si devolvemos otro código, Stripe lo reintentará varias veces.

7.7 El formulario de pago de Stripe.

```
def stripe_payment(request, transaction_id):
    """Muestra el formulario de pago de Stripe."""
    try:
        transaction = Transaction.objects.get(id=transaction_id)
    except Transaction.DoesNotExist:
        return HttpResponse(status=404)

    if not transaction.stripe_client_secret:
        return HttpResponse(
            'Esta transacción no tiene un client secret de Stripe.',
            status=400
        )

    return render(request, 'core/stripe_payment.html', {
        'transaction': transaction,
        'client_secret': transaction.stripe_client_secret,
        'stripe_publishable_key': settings.STRIPE_PUBLISHABLE_KEY,
    })
```

Esta vista muestra la página HTML con el formulario de tarjeta de Stripe.

`Transaction.objects.get(id=transaction_id)`: busca la transacción por su ID. Si no existe devuelve 404.

`if not transaction.stripe_client_secret:`: comprueba que la transacción tiene un client secret. Si no lo tiene significa que no se creó correctamente con Stripe (quizás el proveedor no era Stripe o hubo un error).

`render(request, 'core/stripe_payment.html', {...})`: renderiza la plantilla HTML pasando los datos necesarios. El template recibe la transacción, el client secret y la clave publicable de Stripe.

¿Por qué necesita la clave publicable? En el formulario de pago, el JavaScript del navegador se comunica directamente con Stripe (sin pasar por nuestro servidor) para tokenizar la tarjeta. Para eso necesita la clave publicable, que es segura de exponer en el frontend.

7.8 El pago con Redsys.

```
@csrf_exempt
def redsys_payment(request, transaction_id):
    """Genera el formulario de pago de Redsys."""
    try:
        transaction = Transaction.objects.get(id=transaction_id)
    except Transaction.DoesNotExist:
        return HttpResponse(status=404)

    merchant_params = MerchantParameters(
        DS_MERCHANT_AMOUNT=int(transaction.amount * 100),
        DS_MERCHANT_ORDER=f'P{str(transaction.id).zfill(11)}',
        DS_MERCHANT_MERCHANTCODE=int(settings.REDSYS_MERCHANT_CODE),
        DS_MERCHANT_CURRENCY=978,
        DS_MERCHANT_TRANSACTIONTYPE=0,
        DS_MERCHANT_TERMINAL=int(settings.REDSYS_TERMINAL),
        DS_MERCHANT_MERCHANTURL='https://xxxx.ngrok-free.dev/api/webhooks/redsyst',
        DS_MERCHANT_URLOK='https://xxxx.ngrok-free.dev/transactions/',
        DS_MERCHANT_URLKO='https://xxxx.ngrok-free.dev/transactions/',
    )

    signed = request_payment(
        settings.REDSYS_SECRET_KEY,
        merchant_params
    )

    return render(request, 'core/redsyst_payment.html', {
        'redsyst_url': settings.REDSYS_URL,
        'merchant_parameters': signed.Ds_MerchantParameters,
        'merchant_signature': signed.Ds_Signature,
        'signature_version': signed.Ds_SignatureVersion,
    })
```

Redsys funciona de forma muy diferente a Stripe. En lugar de un formulario propio, redirige al usuario a la web del banco.

DS_MERCHANT_AMOUNT=int(transaction.amount * 100): igual que Stripe, Redsys trabaja en céntimos.

DS_MERCHANT_ORDER=f'P{str(transaction.id).zfill(11)}': el número de pedido que Redsys usa para identificar el pago. Debe ser único y de máximo 12 caracteres. Usamos la letra "P" seguida del ID de la transacción con ceros a la izquierda. **zfill(11)** rellena con ceros hasta 11 caracteres. Así el ID 1 se convierte en **P000000000001**.

DS_MERCHANT_CURRENCY=978: el código numérico ISO 4217 del euro. 978 = EUR.

DS_MERCHANT_TRANSACTIONTYPE=0: tipo de transacción. 0 = autorización estándar.

DS_MERCHANT_MERCHANTURL: URL donde Redsys enviará la notificación cuando el pago se complete. Debe ser una URL pública accesible desde internet, por eso usamos ngrok en desarrollo.

DS_MERCHANT_URLOK y **DS_MERCHANT_URLKO**: URLs donde Redsys redirigirá al usuario después del pago. URLOK para pagos exitosos, URLKO para pagos fallidos.

signed = request_payment(settings.REDSYS_SECRET_KEY, merchant_params): la librería pyredsys firma todos los parámetros con la clave secreta usando el algoritmo HMAC-SHA256. Devuelve un objeto con tres campos: **Ds_MerchantParameters** (los parámetros en base64), **Ds_Signature** (la firma) y **Ds_SignatureVersion**.

La firma es fundamental para la seguridad: Redsys verifica que los parámetros no han sido manipulados comparando la firma que enviamos con la que ellos calculan con la misma clave secreta.

7.9 El webhook de Redsys.

```
@csrf_exempt
def redsys_webhook(request):
    """Recibe y procesa las notificaciones de Redsys."""
    if request.method == 'POST':
        try:
            parameters = request.POST.get('Ds_MerchantParameters')
            signature = request.POST.get('Ds_Signature')
            version = request.POST.get('Ds_SignatureVersion')

            notification = validate_notification(
                settings.REDSYS_SECRET_KEY,
                parameters,
                signature,
                version
            )
            response_code = notification.Ds_Response
            order = notification.Ds_Order.lstrip('P').lstrip('0') or '0'

            if response_code < 100:
                Transaction.objects.filter(id=order).update(
                    status='completed'
                )
            else:
                Transaction.objects.filter(id=order).update(
                    status='failed',
                    incident_type='error_conexion'
                )
        except SignatureVerificationError:
            return HttpResponse(status=400)
        except Exception:
            return HttpResponse(status=400)

    return HttpResponse(status=200)
```

request.POST.get('Ds_MerchantParameters'): Redsys envía los datos como formulario HTML (application/x-www-form-urlencoded), no como JSON. **request.POST** contiene esos datos.

validate_notification(settings.REDSYS_SECRET_KEY, parameters, signature, version): verifica la firma de la notificación. Si alguien intenta enviarnos una notificación falsa, la firma no coincidirá y se lanzará **SignatureVerificationError**. El objeto que devuelve es un **NotificationParameters** con todos los campos de la respuesta de Redsys como atributos.

response_code = notification.Ds_Response: código de respuesta de Redsys. Los códigos menores de 100 indican éxito. Los códigos de 100 en adelante indican error.

order = notification.Ds_Order.lstrip('P').lstrip('0') or '0': recupera el número de pedido que enviamos. Le quitamos la "P" del principio y los ceros con **rstrip**. Así **P000000000001** se convierte en **1**, que es el ID de nuestra transacción. **or '0'** evita que quede vacío si el ID era 0.

7.10 El pago con PayPal.

```
def paypal_payment(request, transaction_id):
    """Crea un pago en PayPal y redirige al usuario."""
    try:
        transaction = Transaction.objects.get(id=transaction_id)
    except Transaction.DoesNotExist:
        return HttpResponse(status=404)

    paypalrestsdk.configure({
        'mode': 'sandbox',
        'client_id': settings.PAYPAL_CLIENT_ID,
        'client_secret': settings.PAYPAL_CLIENT_SECRET,
    })

    payment = paypalrestsdk.Payment({
        'intent': 'sale',
        'payer': {'payment_method': 'paypal'},
        'redirect_urls': {
            'return_url': f'http://127.0.0.1:8000/api/pay/paypal/execute/?transaction_id={transaction_id}',
            'cancel_url': 'http://127.0.0.1:8000/transactions/',
        },
        'transactions': [{
            'amount': {
                'total': str(transaction.amount),
                'currency': transaction.currency,
            },
            'description': f'Transacción #{transaction_id}',
        }]
    })

    if payment.create():
        for link in payment.links:
            if link.rel == 'approval_url':
                return HttpResponse(
                    f'<meta http-equiv="refresh" content="0;url={link.href}">',
                    content_type='text/html'
                )
    return HttpResponse('Error al crear el pago en PayPal.', status=400)
```

PayPal funciona diferente a Stripe y Redsys. El flujo es:

1. Nosotros creamos un pago en PayPal via API.
2. PayPal nos devuelve una URL de aprobación.
3. Redirigimos al usuario a esa URL.
4. El usuario aprueba el pago en la web de PayPal.
5. PayPal redirige de vuelta a nuestra `return_url`.
6. Nosotros ejecutamos el pago definitivamente.

paypalrestsdk.configure({...}): configura el SDK con las credenciales. Se hace en cada petición porque las credenciales podrían cambiar entre llamadas (aunque en práctica siempre son las mismas).

'intent': 'sale': tipo de pago. **sale** significa cobrar inmediatamente. La alternativa es **authorize** que reserva el dinero para cobrarlo más tarde.

'payer': {'payment_method': 'paypal'}: el método de pago es la cuenta PayPal del usuario.

'redirect_urls': URLs de retorno. **return_url** es donde PayPal redirige cuando el usuario aprueba el pago. Incluimos el **transaction_id** como parámetro para saber qué transacción actualizar. **cancel_url** es donde redirige si el usuario cancela.

payment.create(): llama a la API de PayPal para crear el pago. Devuelve **True** si tiene éxito, **False** si falla.

payment.links: lista de URLs relacionadas con el pago. Buscamos la que tiene **rel == 'approval_url'** que es la URL donde el usuario debe aprobar el pago.

<meta http-equiv="refresh" content="0;url=...">: redirige al usuario a la URL de PayPal inmediatamente. Es una redirección HTML instantánea.

7.11 La ejecución del pago PayPal.

```
def paypal_execute(request):
    """Ejecuta el pago de PayPal tras la aprobación del usuario."""
    payment_id = request.GET.get('paymentId')
    payer_id = request.GET.get('PayerID')
    transaction_id = request.GET.get('transaction_id')

    paypalrestsdk.configure({
        'mode': 'sandbox',
        'client_id': settings.PAYPAL_CLIENT_ID,
        'client_secret': settings.PAYPAL_CLIENT_SECRET,
    })

    payment = paypalrestsdk.Payment.find(payment_id)

    if payment.execute({'payer_id': payer_id}):
        Transaction.objects.filter(id=transaction_id).update(
            status='completed'
        )
    else:
        Transaction.objects.filter(id=transaction_id).update(
            status='failed',
            incident_type='error_conexion'
        )

    from django.shortcuts import redirect
    return redirect('/transactions/')
```

Cuando el usuario aprueba el pago en PayPal, PayPal redirige a nuestra `return_url` con dos parámetros en la URL: `paymentId` y `PayerID`.

`payment_id = request.GET.get('paymentId')`: el ID del pago que PayPal generó. `request.GET` contiene los parámetros de la URL.

`payer_id = request.GET.get('PayerID')`: el ID del usuario de PayPal que aprobó el pago. Necesario para ejecutar el pago.

`transaction_id = request.GET.get('transaction_id')`: el ID de nuestra transacción que enviamos en la `return_url`.

`payment = paypalrestsdk.Payment.find(payment_id)`: recupera el pago de la API de PayPal por su ID.

`payment.execute({'payer_id': payer_id})`: ejecuta el cobro definitivo. Hasta este momento el pago estaba aprobado pero no cobrado. Con `execute` se hace el cargo real en la cuenta del usuario.

`redirect('/transactions/')`: redirige al usuario a la lista de transacciones para que vea el estado actualizado.

8. Las vistas del frontend. El panel de gestión.

Las vistas del frontend están en `frontend/views.py`. A diferencia de las vistas de la API que devuelven JSON, estas vistas devuelven páginas HTML completas para que el usuario las vea en el navegador.

8.1 La arquitectura del frontend.

El frontend no accede directamente a la base de datos. En cambio, llama a la propia API REST del backend usando la librería `requests`. Esto puede parecer redundante pero tiene ventajas importantes:

- El frontend y el backend están completamente desacoplados.
- Si mañana quisieras hacer una app móvil, usaría la misma API.
- Las validaciones se aplican siempre, independientemente de quién llame a la API.
- Es más fácil de mantener y escalar.

El flujo es:

Navegador → frontend/views.py → API REST → Base de datos

8.2 Los imports y la configuración base.

```
import requests
from django.shortcuts import render, redirect
from django.contrib import messages
from django.core.paginator import Paginator

API_BASE = 'http://127.0.0.1:8000/api'
```

import requests: la librería para hacer peticiones HTTP. La usamos para llamar a la API.

from django.shortcuts import render, redirect: `render` devuelve una página HTML renderizada. `redirect` dirige al usuario a otra URL.

from django.contrib import messages: sistema de mensajes flash de Django. Permite mostrar mensajes de éxito o error que aparecen una sola vez y desaparecen.

from django.core.paginator import Paginator: paginador de Django. Divide una lista larga en páginas.

API_BASE = 'http://127.0.0.1:8000/api': la URL base de la API. Definida como constante para no repetirla en cada función.

8.3 La función auxiliar **get_token**.

```
def get_token(request):  
    """Obtiene el token de sesión del usuario."""  
    return request.session.get('token')
```

Esta pequeña función es usada por todas las vistas. Recupera el token de autenticación guardado en la sesión del usuario.

¿Qué es la sesión? Django mantiene una sesión por usuario. Es como una memoria temporal que persiste entre peticiones. El token se guarda en la sesión cuando el usuario hace login y se elimina cuando hace logout.

`request.session.get('token')` devuelve el token si existe o **None** si no existe. Todas las vistas comprueban si hay token y si no lo hay redirigen al login.

8.4 La vista **login**.

```
def login_view(request):
    """Vista de login."""
    if request.method == 'POST':
        username = request.POST.get('username')
        password = request.POST.get('password')
        response = requests.post(
            f'{API_BASE}/auth/login/',
            json={'username': username, 'password': password}
        )
        if response.status_code == 200:
            request.session['token'] = response.json()['token']
            request.session['username'] = username
            return redirect('dashboard')
        else:
            messages.error(request, 'Usuario o contraseña incorrectos.')
    return render(request, 'frontend/login.html')
```

Esta vista gestiona dos situaciones:

Cuando el usuario accede por primera vez (GET): simplemente renderiza el formulario de login.

if request.method == 'POST': es False, así que salta directamente al **return render(...)**.

Cuando el usuario envía el formulario (POST), procesa las credenciales:

1. **request.POST.get('username')**: obtiene el usuario del formulario HTML
2. **request.POST.get('password')**: obtiene la contraseña
3. **requests.post(f'{API_BASE}/auth/login/', json={...})**: llama al endpoint de login de la API enviando las credenciales en formato JSON
4. Si la respuesta es 200 (éxito):
 - **response.json()['token']**: extrae el token de la respuesta JSON
 - **request.session['token'] = ...**: guarda el token en la sesión
 - **request.session['username'] = username**: guarda el nombre de usuario para mostrarlo en la barra lateral
 - **return redirect('dashboard')**: redirige al dashboard
5. Si la respuesta no es 200 (credenciales incorrectas):
 - **messages.error(...)**: añade un mensaje de error que se mostrará en la página

8.5 La vista **logout**.

```
def logout_view(request):  
    """Vista de logout."""  
    request.session.flush()  
    return redirect('login')
```

Muy simple. `request.session.flush()` elimina todos los datos de la sesión (token, username...) y crea una sesión nueva vacía. Luego redirige al login.

8.6 La vista **dashboard**.

```
def dashboard(request):
    """Vista del dashboard con resumen de estadísticas."""
    token = get_token(request)
    if not token:
        return redirect('login')

    headers = {'Authorization': f'Token {token}'}
    providers = requests.get(f'{API_BASE}/providers/', headers=headers).json()
    transactions = requests.get(
        f'{API_BASE}/transactions/', headers=headers
    ).json()

    stats = {
        'total_providers': len(providers),
        'total_transactions': len(transactions),
        'pending': sum(1 for t in transactions if t['status'] == 'pending'),
        'completed': sum(1 for t in transactions if t['status'] == 'completed'),
        'failed': sum(1 for t in transactions if t['status'] == 'failed'),
    }

    return render(request, 'frontend/dashboard.html', {
        'stats': stats,
        'username': request.session.get('username'),
    })
```

token = get_token(request): obtiene el token de la sesión.

if not token: return redirect('login'): si no hay token el usuario no está autenticado. Lo redirigimos al login. Este patrón se repite en todas las vistas protegidas.

headers = {'Authorization': f'Token {token}'}: prepara la cabecera de autenticación que hay que incluir en todas las peticiones a la API.

requests.get(f'{API_BASE}/providers/', headers=headers).json(): llama a la API para obtener los proveedores y convierte la respuesta JSON a una lista de diccionarios Python.

stats = {...}: calcula las estadísticas:

- **len(providers)**: número total de proveedores.
- **len(transactions)**: número total de transacciones.
- **sum(1 for t in transactions if t['status'] == 'pending')**: cuenta las transacciones pendientes usando una expresión generadora. Por cada transacción donde el status es 'pending', suma 1.

return render(request, 'frontend/dashboard.html', {'stats': stats, 'username': ...}): renderiza el template pasándole las estadísticas y el nombre de usuario. Estos datos estarán disponibles en el HTML como `{{ stats.pending }}`, `{{ username }}`, etc.

8.7 La vista **providers**.

```
def providers(request):
    """Vista de gestión de proveedores."""
    token = get_token(request)
    if not token:
        return redirect('login')

    headers = {'Authorization': f'Token {token}'}

    if request.method == 'POST':
        data = {
            'name': request.POST.get('name'),
            'api_key': request.POST.get('api_key'),
            'environment': request.POST.get('environment'),
        }
        requests.post(f'{API_BASE}/providers/', json=data, headers=headers)
        return redirect('providers')

    provider_list = requests.get(
        f'{API_BASE}/providers/', headers=headers
    ).json()
    return render(request, 'frontend/providers.html', {
        'providers': provider_list,
        'username': request.session.get('username'),
    })
```

Esta vista gestiona dos situaciones:

GET: mostrar la lista de proveedores:

1. Llama a la API para obtener todos los proveedores.
2. Renderiza el template con la lista.

POST: crear un nuevo proveedor:

1. Recoge los datos del formulario HTML.
2. Llama a la API con POST para crear el proveedor.
3. Redirige a la misma página para que se recargue con el nuevo proveedor.

El patrón POST → redirect es importante. Si después de crear un proveedor no redirigiéramos, el usuario podría recargar la página y volver a enviar el formulario, creando el proveedor dos veces.

8.8 La vista **transactions**, la más compleja.

```
def transactions(request):
    """Vista de gestión de transacciones con filtros."""
    token = get_token(request)
    if not token:
        return redirect('login')

    headers = {'Authorization': f'Token {token}'}
    provider_list = requests.get(
        f'{API_BASE}/providers/', headers=headers
    ).json()

    if request.method == 'POST':
        data = {
            'provider': request.POST.get('provider'),
            'amount': request.POST.get('amount'),
            'currency': request.POST.get('currency'),
            'status': 'pending',
        }
        requests.post(
            f'{API_BASE}/transactions/', json=data, headers=headers
        )
        return redirect('transactions')

    params = {}
    if request.GET.get('status'):
        params['status'] = request.GET.get('status')
    if request.GET.get('provider'):
        params['provider'] = request.GET.get('provider')

    ordering = request.GET.get('ordering', '-created_at')
    params['ordering'] = ordering

    transaction_list = requests.get(
        f'{API_BASE}/transactions/', headers=headers, params=params
    ).json()

    paginator = Paginator(transaction_list, 10)
    page_number = request.GET.get('page', 1)
    page_obj = paginator.get_page(page_number)

    return render(request, 'frontend/transactions.html', {
        'transactions': page_obj,
        'page_obj': page_obj,
        'providers': provider_list,
        'username': request.session.get('username'),
        'selected_status': request.GET.get('status', ''),
        'selected_provider': request.GET.get('provider', ''),
        'ordering': ordering,
    })
```

Esta vista es más compleja porque gestiona filtros, ordenación y paginación.

Los filtros: se leen de los parámetros GET de la URL:

```
params = {}
if request.GET.get('status'):
    params['status'] = request.GET.get('status')
if request.GET.get('provider'):
    params['provider'] = request.GET.get('provider')
```

Si la URL es `/transactions/?status=pending&provider=1`, `params` será `{'status': 'pending', 'provider': '1'}`. Estos parámetros se pasan a la API que los usa para filtrar.

La ordenación:

```
ordering = request.GET.get('ordering', '-created_at')
params['ordering'] = ordering
```

`request.GET.get('ordering', '-created_at')` lee el parámetro `ordering` de la URL. Si no existe usa `-created_at` como valor por defecto (más recientes primero). Este parámetro se pasa a la API que ordena los resultados.

La paginación:

```
paginator = Paginator(transaction_list, 10)
page_number = request.GET.get('page', 1)
page_obj = paginator.get_page(page_number)
```

`Paginator(transaction_list, 10)` divide la lista en páginas de 10 elementos. `get_page(page_number)` devuelve la página solicitada. Si `page_number` es inválido (ej. una letra), `get_page` devuelve la primera página sin lanzar error.

`page_obj` tiene atributos especiales:

- `page_obj.has_previous`: si hay página anterior
- `page_obj.has_next`: si hay página siguiente
- `page_obj.previous_page_number`: número de la página anterior
- `page_obj.next_page_number`: número de la página siguiente
- `page_obj.paginator.page_range`: rango de números de página

8.9 Las vistas **update_transaction** y **delete_provider**.

```
def update_transaction(request, transaction_id):
    """Actualiza el estado e incidencia de una transacción."""
    token = get_token(request)
    if not token:
        return redirect('login')

    if request.method == 'POST':
        headers = {'Authorization': f'Token {token}'}
        data = {
            'provider': request.POST.get('provider'),
            'amount': request.POST.get('amount'),
            'currency': request.POST.get('currency'),
            'status': request.POST.get('status'),
            'incident_type': request.POST.get('incident_type', ''),
        }
        requests.put(
            f'{API_BASE}/transactions/{transaction_id}/',
            json=data,
            headers=headers
        )
    return redirect('transactions')
```

El modal de edición en el frontend envía un formulario POST con todos los datos de la transacción. Esta vista recoge esos datos y llama a la API con PUT para actualizar.

requests.put(...): PUT es el método HTTP para actualizar un recurso completo. La URL incluye el ID de la transacción: **/api/transactions/1/**.

```
def delete_provider(request, provider_id):
    """Elimina un proveedor."""
    token = get_token(request)
    if not token:
        return redirect('login')

    if request.method == 'POST':
        headers = {'Authorization': f'Token {token}'}
        requests.delete(
            f'{API_BASE}/providers/{provider_id}/',
            headers=headers
        )
    return redirect('providers')
```

Similar pero usa **requests.delete(...)**. El botón de eliminar en el frontend está dentro de un formulario POST con confirmación JavaScript (**confirm('¿Eliminar?')**).

¿Por qué POST en lugar de DELETE directamente desde el formulario HTML? Porque los formularios HTML estándar solo soportan GET y POST. Para usar DELETE habría que usar JavaScript fetch. El patrón POST a una URL de eliminación es una solución habitual.

9. Las plantillas HTML: El aspecto visual.

Las plantillas están en `frontend/templates/frontend/`. Son archivos HTML con etiquetas especiales de Django que permiten mostrar datos dinámicos, hacer bucles y condiciones directamente en el HTML.

9.1 ¿Qué es el sistema de plantillas de Django?

Django tiene su propio lenguaje de plantillas. Añade dos tipos de etiquetas al HTML estándar:

Variables `{{ variable }}`: muestran el valor de una variable:

```
<td>{{ t.amount }}</td>
```

Etiquetas `{% etiqueta %}`: ejecutan lógica:

```
{% for t in transactions %}
{% if t.status == 'completed' %}
{% endif %}
{% endfor %}
```

Filtros `{{ variable|filtro }}`: transforman valores:

```
{{ t.created_at|slice:"10" }} <!-- muestra solo los 10 primeros caracteres -->
```

Django procesa estas etiquetas en el servidor y devuelve HTML puro al navegador. El usuario nunca ve las etiquetas de Django.

9.2 La plantilla base: **base.html**.

Esta es la plantilla madre de la que heredan todas las demás. Define la estructura común: barra lateral, cabecera, área de contenido.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Payment Gateway</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
  <link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.0/font/bootstrap-icons.css" rel="stylesheet">
  <style>
    body { background-color: #f8f9fa; }
    .sidebar {
      min-height: 100vh;
      background: #212529;
      padding-top: 1rem;
    }
    .sidebar a {
      color: #adb5bd;
      text-decoration: none;
      display: block;
      padding: 0.75rem 1.5rem;
    }
    .sidebar a:hover, .sidebar a.active {
      background: #495057;
      color: white;
    }
  </style>
</head>
<body>
<div class="container-fluid">
  <div class="row">
    <!-- Barra lateral -->
    <div class="col-md-2 px-0 sidebar">
      <div class="brand">
        <i class="bi bi-credit-card-2-front"></i> Gateway
      </div>
      <a href="{% url 'dashboard' %}"
        class="{% if request.resolver_match.url_name == 'dashboard' %}active{% endif %}"
        <i class="bi bi-speedometer2"></i> Dashboard
      </a>
      <a href="{% url 'providers' %}"
        class="{% if request.resolver_match.url_name == 'providers' %}active{% endif %}"
        <i class="bi bi-building"></i> Proveedores
      </a>
      <a href="{% url 'transactions' %}"
        class="{% if request.resolver_match.url_name == 'transactions' %}active{% endif %}"
        <i class="bi bi-arrow-left-right"></i> Transacciones
      </a>
      <a href="{% url 'logout' %}">
```

```
        <i class="bi bi-box-arrow-left"></i> Cerrar sesión {{ username }}
    </a>
</div>
<!-- Contenido principal -->
<div class="col-md-10 main-content">
    {% if messages %}
        {% for message in messages %}
            <div class="alert alert-{{ message.tags }}">{{ message }}</div>
        {% endfor %}
    {% endif %}
    {% block content %}{% endblock %}
</div>
</div>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

Partes importantes:

{% url 'dashboard' %}: genera la URL de la vista llamada 'dashboard'. Es mejor que escribir `/` directamente porque si cambias las URLs no tienes que actualizar todos los templates.

{% if request.resolver_match.url_name == 'dashboard' %}active{% endif %}: añade la clase CSS `active` al enlace de la página actual. `request.resolver_match.url_name` devuelve el nombre de la vista actual. Así el menú lateral resalta la página en la que estás.

{% if messages %} ... {% endif %}: muestra los mensajes flash si los hay. Estos son los mensajes de error o éxito que se generan con `messages.error(...)` en las vistas.

{% block content %}{% endblock %}: este es el punto donde las plantillas hijas insertan su contenido. Es el mecanismo de herencia de Django.

Herencia de plantillas: todas las demás plantillas empiezan con:

```
{% extends 'frontend/base.html' %}

{% block content %}
<!-- contenido específico de cada página -->
{% endblock %}
```

{% extends %} le dice a Django qué plantilla base usar. **{% block content %}** reemplaza el bloque del mismo nombre en la base. Así cada página tiene la barra lateral y el estilo común sin repetir código.

9.3 La plantilla de login: **login.html**.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Login: Payment Gateway</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
  <style>
    body {
      background: linear-gradient(135deg, #212529 0%, #495057 100%);
      min-height: 100vh;
      display: flex;
      align-items: center;
      justify-content: center;
    }
  </style>
</head>
<body>
  <div class="login-card">
    <div class="brand">
      <i class="bi bi-credit-card-2-front"></i> Payment Gateway
    </div>
    <p class="text-muted mb-4">Inicia sesión para continuar</p>

    {% if messages %}
      {% for message in messages %}
        <div class="alert alert-danger">{{ message }}</div>
      {% endfor %}
    {% endif %}

    <form method="post">
      {% csrf_token %}
      <div class="mb-3">
        <label class="form-label">Usuario</label>
        <input type="text" name="username" class="form-control" required autofocus>
      </div>
      <div class="mb-3">
        <label class="form-label">Contraseña</label>
        <input type="password" name="password" class="form-control" required>
      </div>
      <button type="submit" class="btn btn-dark w-100">
        <i class="bi bi-box-arrow-in-right"></i> Entrar
      </button>
    </form>
  </div>
</body>
</html>
```

Esta plantilla no extiende `base.html` porque el login no tiene barra lateral.

`{% csrf_token %}`: este es obligatorio en todos los formularios POST de Django. Genera un campo oculto con un token único que Django verifica al recibir el formulario. Protege contra ataques CSRF (Cross-Site Request Forgery) donde un sitio malicioso podría enviar formularios en nombre del usuario sin que este lo sepa.

`method="post"`: el formulario envía los datos por POST, no por GET. Importante para contraseñas: GET incluiría la contraseña en la URL visible.

`required`: atributo HTML5 que hace que el navegador no envíe el formulario si el campo está vacío.

`autofocus`: el cursor aparece automáticamente en el campo de usuario al cargar la página.

9.4 La plantilla del dashboard: **dashboard.html**.

```
{% extends 'frontend/base.html' %}
{% block content %}
<h2 class="mb-4"><i class="bi bi-speedometer2"></i> Dashboard</h2>
<div class="row g-4 mb-4">
  <div class="col-md-3">
    <div class="card stat-card bg-dark text-white">
      <div class="card-body">
        <div class="d-flex justify-content-between align-items-center">
          <div>
            <div class="text-white-50 small fw-bold text-uppercase mb-1">Proveedores</div>
            <div class="fs-1 fw-bold">{{ stats.total_providers }}</div>
          </div>
          <i class="bi bi-building fs-1 opacity-25"></i>
        </div>
      </div>
    </div>
  </div>
  <!-- más tarjetas... -->
</div>
<!-- Gráfico -->
<div class="row g-4 mt-2">
  <div class="col-md-6">
    <div class="card stat-card">
      <div class="card-body">
        <h5 class="card-title">Transacciones por estado</h5>
        <canvas id="transactionChart" height="200"></canvas>
      </div>
    </div>
  </div>
</div>
</div>
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
<script>
const ctx = document.getElementById('transactionChart').getContext('2d');
new Chart(ctx, {
  type: 'doughnut',
  data: {
    labels: ['Pendientes', 'Completadas', 'Fallidas'],
    datasets: [{
      data: [{ stats.pending }, { stats.completed }, { stats.failed }],
      backgroundColor: ['#ffc107', '#198754', '#dc3545'],
      borderWidth: 2
    }]
  },
  options: {
    responsive: true,
    plugins: { legend: { position: 'bottom' } }
  }
});
</script>
{% endblock %}
```

`{{ stats.total_providers }}`: muestra el número de proveedores que pasó la vista.

`{{ stats.pending }}` dentro del JavaScript: esto es muy interesante. Django procesa las etiquetas antes de enviar el HTML al navegador. Así los valores Python se convierten en números JavaScript directamente. El resultado final en el navegador sería:

```
data: [10, 3, 1],
```

Chart.js: librería de gráficos que se carga desde CDN. Con muy poco código genera gráficos interactivos. `type: 'doughnut'` crea un gráfico de rosca. `backgroundColor` define los colores de cada segmento.

9.5 La plantilla de transacciones: **transactions.html**.

Esta es la más compleja. Tiene el formulario de creación, los filtros, la tabla con paginación, los botones de pago y el modal de edición.

El formulario de creación:

```
<form method="post">
  {% csrf_token %}
  <input type="hidden" name="action" value="create">
  <div class="mb-3">
    <label class="form-label">Proveedor</label>
    <select name="provider" class="form-select" required>
      {% for provider in providers %}
        <option value="{{ provider.id }}">{{ provider.name }}</option>
      {% endfor %}
    </select>
  </div>
  <div class="mb-3">
    <label class="form-label">Importe</label>
    <div class="input-group">
      <input type="number" name="amount" class="form-control"
        step="0.01" min="0.01" required>
      <span class="input-group-text">€</span>
    </div>
  </div>
</form>
```

{% for provider in providers %}: bucle que genera una opción del select por cada proveedor. **{{ provider.id }}** es el valor que se envía al servidor. **{{ provider.name }}** es lo que ve el usuario.

step="0.01": permite importes con decimales. Sin esto solo se aceptarían números enteros.

min="0.01": validación HTML5 que impide importes negativos o cero.

Las cabeceras ordenables:

```
<th>
  <a href="?ordering={% if ordering == 'amount' %}-amount{% else %}amount{% endif
%}&status={{ selected_status }}&provider={{ selected_provider }}"
  class="text-decoration-none text-dark">
    Importe
    {% if ordering == 'amount' %}<i class="bi bi-arrow-up"></i>
    {% elif ordering == '-amount' %}<i class="bi bi-arrow-down"></i>
    {% endif %}
  </a>
</th>
```

Esto implementa la ordenación al hacer click en la cabecera. La lógica es:

- Si actualmente está ordenado por **amount** (ascendente), el enlace cambia a **-amount** (descendente): orden alternado
- Si está ordenado por cualquier otra cosa, el enlace ordena por **amount** (ascendente)
- El icono de flecha indica la dirección actual
- Los parámetros **status** y **provider** se mantienen para no perder los filtros activos

Los botones de pago:

```
<td class="d-flex gap-1 align-items-center">
  {% if t.status == 'pending' %}
    {% if t.provider_name == 'Redsys' %}
      <a href="/api/pay/redsys/{{ t.id }}"
        class="btn btn-sm btn-warning text-dark">
        <i class="bi bi-bank"></i> Pagar
      </a>
    {% elif t.provider_name == 'Stripe' %}
      <a href="/api/pay/stripe/{{ t.id }}"
        class="btn btn-sm btn-primary">
        <i class="bi bi-credit-card"></i> Pagar
      </a>
    {% elif t.provider_name == 'PayPal' %}
      <a href="/api/pay/paypal/{{ t.id }}"
        class="btn btn-sm btn-info text-white">
        <i class="bi bi-paypal"></i> Pagar
      </a>
    {% endif %}
  {% endif %}
```

Los botones de pago solo aparecen cuando **t.status == 'pending'**. No tiene sentido pagar una transacción ya completada o fallida. El color y el icono cambian según el proveedor para que el usuario identifique visualmente con qué pasarela va a pagar.

La paginación:

```
{% if page_obj.has_other_pages %}
<div class="card-body border-top">
  <nav>
    <ul class="pagination pagination-sm mb-0 justify-content-center">
      {% if page_obj.has_previous %}
        <li class="page-item">
          <a class="page-link" href="?
page={{ page_obj.previous_page_number }}&status={{ selected_status }}&provider={{ selected_provid
er }}">
            <i class="bi bi-chevron-left"></i>
          </a>
        </li>
      {% endif %}
      {% for num in page_obj.paginator.page_range %}
        <li class="page-item {% if page_obj.number == num %}active{% endif %}">
          <a class="page-link" href="?
page={{ num }}&status={{ selected_status }}&provider={{ selected_provider }}">{{ num }}</a>
        </li>
      {% endfor %}
      {% if page_obj.has_next %}
        <li class="page-item">
          <a class="page-link" href="?
page={{ page_obj.next_page_number }}&status={{ selected_status }}&provider={{ selected_provider }}
">
            <i class="bi bi-chevron-right"></i>
          </a>
        </li>
      {% endif %}
    </ul>
  </nav>
</div>
{% endif %}
```

La paginación solo se muestra si hay más de una página (**has_other_pages**). Cada enlace de página incluye los filtros actuales para no perderlos al cambiar de página.

El modal de edición:

```

<div class="modal fade" id="updateModal" tabindex="-1">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header bg-dark text-white">
        <h5 class="modal-title">Actualizar transacción</h5>
        <button type="button" class="btn-close btn-close-white" data-bs-dismiss="modal"></button>
      </div>
      <form id="updateForm" method="post" action="">
        {% csrf_token %}
        <input type="hidden" name="provider" id="modal_provider">
        <input type="hidden" name="amount" id="modal_amount">
        <input type="hidden" name="currency" id="modal_currency">
        <div class="modal-body">
          <div class="mb-3">
            <label class="form-label">Estado</label>
            <select name="status" id="modal_status" class="form-select">
              <option value="pending">Pending</option>
              <option value="completed">Completed</option>
              <option value="failed">Failed</option>
            </select>
          </div>
          <div class="mb-3">
            <label class="form-label">Tipo de incidencia</label>
            <select name="incident_type" class="form-select">
              <option value="">Sin incidencia</option>
              <option value="impago">Impago</option>
              <option value="error_conexion">Error de conexión</option>
              <option value="devolucion">Devolución</option>
            </select>
          </div>
        </div>
        <div class="modal-footer">
          <button type="button" class="btn btn-secondary" data-bs-dismiss="modal">Cancelar</button>
          <button type="submit" class="btn btn-dark">Guardar</button>
        </div>
      </form>
    </div>
  </div>
</div>

```

Un modal es una ventana emergente. Bootstrap incluye todo el sistema de modales. El modal tiene campos ocultos (**type="hidden"**) para el proveedor, importe y moneda porque el formulario PUT necesita todos los campos de la transacción, no solo los que el usuario puede cambiar.

El JavaScript del modal:

```
function openModal(id, provider, amount, currency, status) {  
    document.getElementById('updateForm').action = '/transactions/' + id + '/update/';  
    document.getElementById('modal_provider').value = provider;  
    document.getElementById('modal_amount').value = amount;  
    document.getElementById('modal_currency').value = currency;  
    document.getElementById('modal_status').value = status;  
    new bootstrap.Modal(document.getElementById('updateModal')).show();  
}
```

Esta función se llama cuando el usuario hace click en el lápiz de una transacción. Rellena el modal con los datos actuales de esa transacción y lo muestra. El botón del lápiz en la tabla llama a esta función:

```
<button class="btn btn-sm btn-outline-secondary"  
    onclick="openModal('{{ t.id }}', '{{ t.provider }}', '{{ t.amount }}', '{{ t.currency }}', '{{ t.status }}')">  
    <i class="bi bi-pencil"></i>  
</button>
```

Django rellena los valores de Python en los parámetros de la función JavaScript antes de enviar el HTML al navegador.

9.6 Las plantillas de pago.

core/templates/core/stripe_payment.html: el formulario de tarjeta de Stripe:

```
<script src="https://js.stripe.com/v3/"></script>
<script>
  const stripe = Stripe('{{ stripe_publishable_key }}');
  const elements = stripe.elements();
  const card = elements.create('card');
  card.mount('#card-element');

  document.getElementById('pay-btn').addEventListener('click', async () => {
    const { error, paymentIntent } = await stripe.confirmCardPayment(
      '{{ client_secret }}',
      { payment_method: { card: card } }
    );

    if (error) {
      document.getElementById('card-errors').textContent = error.message;
    } else if (paymentIntent.status === 'succeeded') {
      window.location.href = '/transactions/';
    }
  });
</script>
```

Este template carga el SDK de JavaScript de Stripe. `Stripe('{{ stripe_publishable_key }}')` inicializa Stripe con la clave publicable. `stripe.elements()` crea el sistema de componentes de Stripe. `elements.create('card')` crea un campo de tarjeta seguro que Stripe gestiona completamente: nunca vemos los datos de la tarjeta en nuestro servidor.

`stripe.confirmCardPayment('{{ client_secret }}', ...)`: confirma el pago con el client secret que guardamos en la base de datos. Stripe se comunica directamente con sus servidores desde el navegador del usuario. Si tiene éxito, redirigimos a la lista de transacciones.

core/templates/core/redsys_payment.html: el formulario de Redsys:

```
<form id="redsys_form" action="{{ redsys_url }}" method="post">
  <input type="hidden" name="Ds_SignatureVersion" value="{{ signature_version }}">
  <input type="hidden" name="Ds_MerchantParameters" value="{{ merchant_parameters }}">
  <input type="hidden" name="Ds_Signature" value="{{ merchant_signature }}">
  <button type="submit">Pagar ahora</button>
</form>
<script>
  document.getElementById('redsys_form').submit();
</script>
```

Este template es más simple. Genera un formulario con los tres campos que Redsys exige y lo envía automáticamente con JavaScript. El usuario ni siquiera ve el formulario: aparece la página de Redsys directamente.

El formulario se envía a `{{ redsys_url }}` que es la URL del TPV virtual de Redsys (<https://sis-t.redsys.es:25443/sis/realizarPago>).

10. Configuración y uso de cada pasarela de pago.

Esta es la parte más práctica del proyecto. Vamos a explicar en detalle qué hay que hacer para conectar con cada pasarela, cómo funciona técnicamente y qué ocurre en cada paso del proceso de pago.

10.1 Stripe: Configuración completa.

¿Qué es Stripe?

Stripe es una empresa americana que ofrece infraestructura de pagos para internet. Es muy popular porque tiene una API muy bien documentada, SDKs oficiales para casi todos los lenguajes y un excelente entorno de pruebas (sandbox) que no requiere dinero real.

Paso 1: Crear una cuenta en Stripe.

1. Ve a <https://stripe.com> y haz click en **Start now**.
2. Introduce tu email, nombre y contraseña.
3. Verifica tu email.
4. Entra al dashboard.

Por defecto Stripe abre en **modo Live** (producción real). Para desarrollo siempre hay que usar **modo Test**. El toggle está en la esquina superior derecha del dashboard.

Paso 2: Obtener las claves de API.

En el dashboard de Stripe ve a **Developers** → **API keys** → **Test mode**. Verás dos claves:

- **Publishable key** (`pk_test_...`): clave pública. Se puede exponer en el frontend porque solo sirve para inicializar el formulario de pago.
- **Secret key** (`sk_test_...`): clave privada. Nunca debe salir del servidor. Con esta clave se pueden crear pagos, hacer reembolsos, etc.

Guárdalas en el `.env`:

```
STRIPE_SECRET_KEY=sk_test_51TTIkACQLY7w...  
STRIPE_PUBLISHABLE_KEY=pk_test_51TTIkACQLY7w...
```

Paso 3: Instalar el SDK de Stripe.

```
pip install stripe
```

El SDK se instala como cualquier librería Python. En el código lo importamos con `import stripe` y lo configuramos con `stripe.api_key = settings.STRIPE_SECRET_KEY`.

Paso 4: Configurar los webhooks con Stripe CLI.

Los webhooks son notificaciones que Stripe envía a nuestra API cuando ocurre algo (un pago se completa, falla, etc.). El problema en desarrollo es que nuestro servidor está en **localhost** y Stripe no puede alcanzarlo desde internet.

La Stripe CLI soluciona esto creando un túnel local.

Instalación en Linux:

```
curl -s https://packages.stripe.dev/api/security/keypair/stripe-cli-gpg/public | gpg --dearmor | sudo tee /usr/share/keyrings/stripe.gpg
echo "deb [signed-by=/usr/share/keyrings/stripe.gpg] https://packages.stripe.dev/stripe-cli-debian-local stable main" | sudo tee /etc/apt/sources.list.d/stripe.list
sudo apt update && sudo apt install stripe
```

Instalación en Windows:

Descarga el instalador desde <https://github.com/stripe/stripe-cli/releases/latest>: busca el archivo **stripe_X.X.X_windows_x86_64.zip**, descomprímelo y coloca **stripe.exe** en una carpeta accesible.

Autenticación:

```
stripe login
```

Abre el navegador para autenticarte con tu cuenta de Stripe. Solo hay que hacerlo una vez.

Arrancar el listener:

```
stripe listen --forward-to http://127.0.0.1:8000/api/webhooks/stripe/
```

Esto muestra:

```
> Ready! Your webhook signing secret is whsec_be7f582565...
```

Guarda ese **whsec_...** en el **.env**:

```
STRIPE_WEBHOOK_SECRET=whsec_be7f582565...
```

Importante: el webhook secret cambia cada vez que ejecutas **stripe listen**. Pero si siempre usas la misma sesión de Stripe CLI (sin cerrarla), el secret es el mismo. Si reinicias el ordenador hay que volver a ejecutar **stripe listen** y actualizar el **.env**.

Paso 5: El flujo completo de un pago Stripe.

Cuando el usuario crea una transacción con proveedor Stripe:

1. Creación del PaymentIntent.

```
intent = stripe.PaymentIntent.create(
    amount=int(amount * 100), # 99.99€ → 9999 céntimos
    currency='eur',
    automatic_payment_methods={
        'enabled': True,
        'allow_redirects': 'never'
    }
)
```

Stripe devuelve un objeto con `intent.id` (el identificador del pago en Stripe) y `intent.client_secret` (necesario para confirmar el pago desde el navegador). Los guardamos en la base de datos.

2. El formulario de pago.

El usuario hace click en "Pagar" y ve el formulario de tarjeta. El SDK de Stripe JavaScript crea un campo seguro donde el usuario introduce los datos. Los datos de la tarjeta nunca pasan por nuestro servidor: van directamente a Stripe.

3. Confirmación del pago.

```
const { error, paymentIntent } = await stripe.confirmCardPayment(
  '{{ client_secret }}',
  { payment_method: { card: card } }
);
```

El JavaScript del navegador se comunica directamente con Stripe para confirmar el pago usando el `client_secret`.

4. El webhook.

Stripe envía un evento `payment_intent.succeeded` a `http://127.0.0.1:8000/api/webhooks/stripe/`. La Stripe CLI intercepta este evento y lo reenvía a nuestro servidor local. Nuestra API verifica la firma y actualiza el estado de la transacción a `completed`.

Tarjetas de prueba de Stripe

Stripe tiene tarjetas de prueba predefinidas:

Número	Resultado
4242 4242 4242 4242	Pago exitoso
4000 0000 0000 0002	Tarjeta rechazada
4000 0025 0000 3155	Requiere autenticación 3DS

La fecha de caducidad puede ser cualquier fecha futura y el CVC cualquier número de 3 dígitos.

10.2 Redsys: Configuración completa.

¿Qué es Redsys?

Redsys es la plataforma de pagos de los principales bancos españoles: BBVA, CaixaBank, Santander, Bankia, etc. Es la pasarela más usada en España para comercio electrónico. A diferencia de Stripe, no tiene un SDK oficial moderno para Python, por lo que usamos la librería de terceros **pyredsys**.

Credenciales de prueba públicas.

Redsys ofrece un entorno de pruebas con credenciales genéricas que cualquier desarrollador puede usar sin necesidad de cuenta bancaria:

```
REDSYS_SECRET_KEY=sq7HjrUOBfKmC576ILgskD5srU870gj7
REDSYS_MERCHANT_CODE=999008881
REDSYS_TERMINAL=1
REDSYS_URL=https://sis-t.redsys.es:25443/sis/realizarPago
```

Estas credenciales son públicas y están documentadas en la web de Redsys. Solo funcionan en el entorno de pruebas (**sis-t.redsys.es**).

Paso 1: Instalar pyredsys.

```
pip install pyredsys
```

Esta librería instala también **cryptography** y **pydantic** como dependencias, que son necesarias para las operaciones criptográficas y la validación de datos.

Paso 2: Configurar ngrok.

Redsys necesita una URL pública real para enviar las notificaciones de pago. En producción usarías tu dominio real (**https://tutienda.com/api/webhooks/redsys/**). En desarrollo necesitamos ngrok.

Instalación en Linux:

```
sudo snap install ngrok
```

Crear cuenta y obtener authtoken:

1. Ve a <https://ngrok.com> y crea una cuenta gratuita.
2. En el dashboard de ngrok copia tu authtoken.
3. Configúralo:

```
ngrok config add-authtoken TU_AUTHTOKEN
```

Arrancar el túnel:

```
ngrok http 8000
```

ngrok te dará una URL como **https://outscore-footless-pebbly.ngrok-free.dev**. Esta URL es accesible desde cualquier lugar del mundo y reenvía las peticiones a tu servidor local en el puerto 8000.

Importante: en el plan gratuito de ngrok la URL cambia cada vez que reinicias ngrok. Hay que actualizar dos sitios:

En **core/views.py** dentro de **redsys_payment**:

```
DS_MERCHANT_MERCHANTURL='https://NUEVA_URL.ngrok-free.dev/api/webhooks/redsys/',  
DS_MERCHANT_URLOK='https://NUEVA_URL.ngrok-free.dev/transactions/',  
DS_MERCHANT_URLKO='https://NUEVA_URL.ngrok-free.dev/transactions/',
```

En **config/settings.py**:

```
ALLOWED_HOSTS = ['127.0.0.1', 'localhost', '.ngrok-free.dev']  
CSRF_TRUSTED_ORIGINS = ['https://NUEVA_URL.ngrok-free.dev']
```

Paso 3: El sistema de firma HMAC-SHA256.

Este es el aspecto más técnico de Redsys. Para evitar que nadie manipule los parámetros del pago (por ejemplo, cambiar el importe de 100€ a 1€), Redsys usa un sistema de firma digital.

El proceso es:

1. Generamos los parámetros del pago (importe, moneda, número de pedido...).
2. Los convertimos a JSON y los codificamos en Base64.
3. Generamos una clave derivada: **HMAC-SHA256(secret_key, order_number)**.
4. Generamos la firma: **HMAC-SHA512(derived_key, parameters_base64)**.
5. Enviamos los parámetros en Base64 y la firma a Redsys.
6. Redsys hace el mismo proceso con su copia de la clave y compara las firmas.

Si alguien manipula los parámetros (cambia el importe), la firma no coincidirá y Redsys rechazará el pago.

La librería **pyredsys** hace todo esto automáticamente:

```
from pyredsys.payment import request_payment, MerchantParameters

merchant_params = MerchantParameters(
    DS_MERCHANT_AMOUNT=9999,      # 99.99€ en céntimos
    DS_MERCHANT_ORDER='P00000000001',
    DS_MERCHANT_MERCHANTCODE=999008881,
    DS_MERCHANT_CURRENCY=978,     # 978 = EUR
    DS_MERCHANT_TRANSACTIONTYPE=0, # 0 = autorización
    DS_MERCHANT_TERMINAL=1,
    DS_MERCHANT_MERCHANTURL='https://xxx.ngrok-free.dev/api/webhooks/redsys/',
    DS_MERCHANT_URLLOK='https://xxx.ngrok-free.dev/transactions/',
    DS_MERCHANT_URLKO='https://xxx.ngrok-free.dev/transactions/',
)

signed = request_payment('sq7HjrUOBfKmC576ILgskD5srU870gj7', merchant_params)
# signed.Ds_MerchantParameters → parámetros en Base64
# signed.Ds_Signature → firma HMAC-SHA512 en Base64
# signed.Ds_SignatureVersion → 'HMAC_SHA256_V1'
```

Paso 4: El flujo completo de un pago Redsys.

- 1. El usuario hace click en "Pagar".** Nuestro servidor genera el formulario firmado y lo envía al navegador. El formulario apunta a la URL de Redsys y se envía automáticamente con JavaScript.
- 2. El TPV virtual de Redsys.** El usuario ve la página del banco con el formulario de tarjeta. Introduce los datos y confirma. Redsys procesa el pago con el banco.
- 3. Autenticación 3DS.** Redsys redirige al simulador de autenticación bancaria (3D Secure). En el entorno de pruebas aparece una pantalla con tres opciones: "Autenticación con éxito", "Denegar autenticación" y "Autenticación cancelada". En producción real sería un código SMS o una notificación de la app del banco.
- 4. La notificación webhook.** Antes de redirigir al usuario, Redsys envía una notificación POST a **DS_MERCHANT_MERCHANTURL** con el resultado del pago. Nuestra API verifica la firma y actualiza el estado.
- 5. Redirección al usuario.** Redsys redirige al usuario a **DS_MERCHANT_URLLOK** (pago exitoso) o **DS_MERCHANT_URLKO** (pago fallido).

Tarjetas de prueba de Redsys.

Número	Resultado
4548 8120 4940 0004	Pago exitoso con 3DS
4548 8120 4940 0028	Pago denegado

Caducidad: **12/34**, CVV: **123**

10.3 PayPal: Configuración completa.

¿Qué es PayPal?

PayPal es la pasarela de pago más conocida del mundo. Permite pagar con saldo de la cuenta PayPal o con tarjeta de crédito. Muchos usuarios prefieren PayPal porque no tienen que introducir los datos de su tarjeta en cada tienda.

Paso 1: Crear cuenta de desarrollador.

1. Ve a <https://developer.paypal.com>.
2. Haz login con tu cuenta de PayPal normal.
3. PayPal crea automáticamente un entorno de desarrollador vinculado a tu cuenta.

Paso 2: Crear una aplicación sandbox.

1. En el dashboard ve a **Apps & Credentials**.
2. Asegúrate de estar en modo **Sandbox** (toggle arriba).
3. Click en **Create App**.
4. Nombre: cualquiera (ej. **payment-gateway-api**).
5. Tipo: **Merchant** (comerciante que acepta pagos).
6. Click en **Create App**.

PayPal genera dos claves:

- **Client ID** (**AcXYVu . . .**): identificador de la aplicación.
- **Client Secret** (**EHwRMP . . .**): clave secreta.

Guárdalas en el **.env**:

```
PAYPAL_CLIENT_ID=AcXYVu...
PAYPAL_CLIENT_SECRET=EHwRMP...
PAYPAL_URL=https://api-m.sandbox.paypal.com
```

Paso 3: Instalar el SDK de PayPal.

```
pip install paypalrestsdk
```

Este SDK es más antiguo que el de Stripe pero sigue funcionando correctamente para el flujo básico de pagos.

Paso 4: Las cuentas sandbox.

PayPal crea automáticamente cuentas de prueba (sandbox accounts) para simular compradores y vendedores:

En el dashboard de PayPal Developer ve a **Testing Tools** → **Sandbox Accounts**. Verás dos tipos:

- **Business** (negocio/vendedor): es tu cuenta como comerciante.
- **Personal** (particular/comprador): es la cuenta del cliente que compra.

Para probar pagos, inicia sesión en la web de PayPal sandbox con las credenciales de la cuenta **Personal**. Puedes ver las credenciales haciendo click en los tres puntos de la cuenta y seleccionando "View/Edit Account".

Paso 5: El flujo completo de un pago PayPal.

PayPal usa un flujo de tres pasos llamado **Express Checkout**:

1. Crear el pago (Create)

```
payment = paypalrestsdk.Payment({
    'intent': 'sale',
    'payer': {'payment_method': 'paypal'},
    'redirect_urls': {
        'return_url': 'http://127.0.0.1:8000/api/pay/paypal/execute/?transaction_id=1',
        'cancel_url': 'http://127.0.0.1:8000/transactions/',
    },
    'transactions': [{
        'amount': {'total': '99.99', 'currency': 'EUR'},
        'description': 'Transacción #1',
    }]
})
payment.create()
```

PayPal devuelve un objeto con una URL de aprobación. El pago está creado pero pendiente de aprobación del usuario.

2. Aprobar el pago (Approve) El usuario es redirigido a la web de PayPal donde inicia sesión y aprueba el pago. PayPal redirige de vuelta a nuestra **return_url** con dos parámetros: **paymentId** y **PayerID**.

3. Ejecutar el pago (Execute)

```
payment = paypalrestsdk.Payment.find(payment_id)
payment.execute({'payer_id': payer_id})
```

Con **execute** se hace el cargo definitivo en la cuenta del usuario. Si tiene éxito, actualizamos el estado de nuestra transacción a **completed**.

Diferencia con Stripe y Redsys.

A diferencia de Stripe (que tiene webhooks automáticos) y Redsys (que envía una notificación antes de redirigir), PayPal en este flujo no necesita webhook. El propio callback de **return_url** es suficiente para confirmar que el usuario aprobó el pago, y **execute** confirma que el dinero se transfirió correctamente.

10.4 Comparativa de las tres pasarelas

Aspecto	Stripe	Redsys	PayPal
SDK Python oficial	Sí (stripe)	No (pyredsys)	Sí (paypalrestsdk)
Formulario de pago	Propio (en nuestra web)	Web del banco	Web de PayPal
Seguridad	Firma HMAC webhook	Firma HMAC-SHA256	OAuth 2.0
Webhooks	Sí (Stripe CLI en dev)	Sí (ngrok en dev)	No necesario
Tarjetas de prueba	Sí	Sí	Cuentas sandbox
Dificultad de integración	Fácil	Media	Media
Uso en España	Muy común	Muy común (banca)	Común

11. Los tests automáticos.

Los tests están en `core/tests.py` y `frontend/tests.py`. Son programas que verifican automáticamente que el código funciona correctamente. En lugar de probar manualmente cada funcionalidad cada vez que cambias algo, los tests lo hacen por ti en segundos.

11.1 ¿Por qué son importantes los tests?

Imagina que tienes 50 funcionalidades en tu aplicación y cambias una parte del código. Sin tests tendrías que probar manualmente las 50 funcionalidades para asegurarte de que no has roto nada. Con tests ejecutas un comando y en segundos sabes si todo sigue funcionando.

Los tests también documentan el comportamiento esperado del sistema. Leer los tests te dice exactamente cómo debe comportarse cada parte del código.

El proyecto tiene **13 tests** que se ejecutan con:

```
python manage.py test
```

Django crea una base de datos temporal para los tests, ejecuta todos los tests y la destruye al terminar. La base de datos de desarrollo no se ve afectada.

11.2 Los tests de la API: `core/tests.py`

Configuración común: setUp

Ambas clases de tests tienen un método `setUp` que se ejecuta antes de cada test. Prepara el entorno necesario:

```
def setUp(self):
    """Configura el cliente autenticado y un proveedor de prueba."""
    self.client = APIClient()
    self.user = User.objects.create_user(
        username='testuser',
        password='testpass123'
    )
    self.token = Token.objects.create(user=self.user)
    self.client.credentials(
        HTTP_AUTHORIZATION='Token ' + self.token.key
    )
    self.provider = Provider.objects.create(
        name='Stripe',
        api_key='sk_test_123',
        environment='sandbox'
    )
```

APIClient(): cliente de prueba de DRF. Simula peticiones HTTP sin necesidad de un servidor real.

User.objects.create_user(...): crea un usuario de prueba en la base de datos temporal.

Token.objects.create(user=self.user): crea un token de autenticación para ese usuario.

self.client.credentials(HTTP_AUTHORIZATION='Token ' + self.token.key): configura el cliente para que incluya el token en todas las peticiones. Así simulamos un usuario autenticado.

Provider.objects.create(...): crea un proveedor de prueba directamente en la base de datos, sin pasar por la API.

Test 1: Listar proveedores.

```
def test_listar_proveedores(self):
    """GET /api/providers/ debe devolver 200 y la lista de proveedores."""
    response = self.client.get('/api/providers/')
    self.assertEqual(response.status_code, 200)
    self.assertEqual(len(response.data), 1)
```

self.client.get('/api/providers/'): hace una petición GET al endpoint. Como el cliente tiene credenciales configuradas en **setUp**, la petición incluye el token automáticamente.

self.assertEqual(response.status_code, 200): verifica que la respuesta es HTTP 200 (OK). Si no lo es, el test falla con un mensaje de error.

self.assertEqual(len(response.data), 1): verifica que la respuesta contiene exactamente 1 proveedor (el que creamos en **setUp**). **response.data** es la lista de proveedores deserializada.

Test 2: Crear proveedor.

```
def test_crear_proveedor(self):
    """POST /api/providers/ debe crear un proveedor y devolver 201."""
    data = {
        'name': 'PayPal',
        'api_key': 'pk_test_456',
        'environment': 'sandbox'
    }
    response = self.client.post('/api/providers/', data, format='json')
    self.assertEqual(response.status_code, 201)
    self.assertEqual(Provider.objects.count(), 2)
```

`self.client.post('/api/providers/', data, format='json')`: hace una petición POST con los datos en formato JSON.

`self.assertEqual(response.status_code, 201)`: verifica HTTP 201 (Created). La convención REST es usar 201 al crear recursos.

`self.assertEqual(Provider.objects.count(), 2)`: verifica que ahora hay 2 proveedores en la base de datos: el creado en `setUp` y el que acabamos de crear.

Test 3: Acceso sin token.

```
def test_acceso_sin_token(self):
    """GET sin autenticación debe devolver 401."""
    self.client.credentials()
    response = self.client.get('/api/providers/')
    self.assertEqual(response.status_code, 401)
```

`self.client.credentials()`: llama a `credentials()` sin argumentos para limpiar las credenciales. Ahora el cliente no enviará el token.

`self.assertEqual(response.status_code, 401)`: verifica que sin token la API devuelve HTTP 401 (Unauthorized). Este test comprueba que la protección funciona correctamente.

Test 4: Crear transacción.

```
def test_crear_transaccion(self):
    """POST /api/transactions/ debe devolver 201."""
    data = {
        'provider': self.provider.id,
        'amount': '49.99',
        'currency': 'EUR',
        'status': 'pending'
    }
    response = self.client.post(
        '/api/transactions/', data, format='json'
    )
    self.assertEqual(response.status_code, 201)
```

Verifica que se puede crear una transacción correctamente. Usa `self.provider.id` para referenciar el proveedor creado en `setUp`.

Test 5: Importe negativo.

```
def test_importe_negativo(self):
    """POST con importe negativo debe devolver 400."""
    data = {
        'provider': self.provider.id,
        'amount': '-10.00',
        'currency': 'EUR',
        'status': 'pending'
    }
    response = self.client.post(
        '/api/transactions/', data, format='json'
    )
    self.assertEqual(response.status_code, 400)
```

Verifica que la validación del serializer funciona. Un importe negativo debe devolver HTTP 400 (Bad Request). Si el serializer no validara correctamente, se guardaría un importe negativo en la base de datos.

Test 6: Failed sin incidencia.

```
def test_failed_sin_incidencia(self):
    """PUT con status failed sin incident_type debe devolver 400."""
    response = self.client.put(
        f'/api/transactions/{self.transaction.id}/',
        {
            'provider': self.provider.id,
            'amount': '99.99',
            'currency': 'EUR',
            'status': 'failed',
            'incident_type': ''
        },
        format='json'
    )
    self.assertEqual(response.status_code, 400)
```

Verifica la segunda validación del serializer: no se puede marcar una transacción como fallida sin especificar el tipo de incidencia.

Test 7: Filtro por estado.

```
def test_filtro_por_estado(self):
    """GET con ?status=pending debe filtrar correctamente."""
    response = self.client.get('/api/transactions/?status=pending')
    self.assertEqual(response.status_code, 200)
    self.assertTrue(
        all(t['status'] == 'pending' for t in response.data)
    )
```

`all(t['status'] == 'pending' for t in response.data)`: expresión generadora que comprueba que TODAS las transacciones en la respuesta tienen status 'pending'. **`all()`** devuelve True solo si todos los elementos son True.

11.3 Los tests del frontend: frontend/tests.py.

Los tests del frontend son diferentes porque no podemos probar las vistas que llaman a la API directamente: el servidor no está corriendo durante los tests. En cambio probamos el comportamiento de protección de las rutas.

```
class FrontendLoginTests(TestCase):
    """Pruebas para el login del frontend."""

    def setUp(self):
        """Configura el cliente y usuario de prueba."""
        self.client = Client()
        self.user = User.objects.create_user(
            username='testuser',
            password='testpass123'
        )
```

Aquí se usa `Client()` de Django (no `APIClient()` de DRF) porque estas son vistas HTML, no de API.

Test 8: La página de login carga.

```
def test_login_page_carga(self):
    """La página de login debe devolver 200."""
    response = self.client.get('/login/')
    self.assertEqual(response.status_code, 200)
```

Verifica que la página de login existe y es accesible sin autenticación.

Test 9: Login incorrecto.

```
def test_login_incorrecto(self):
    """Login con credenciales incorrectas vuelve al login."""
    response = self.client.post('/login/', {
        'username': 'testuser',
        'password': 'wrongpass'
    })
    self.assertEqual(response.status_code, 200)
```

Verifica que con contraseña incorrecta no se redirige (código 302) sino que se vuelve a mostrar el formulario de login (código 200) con el mensaje de error.

Tests 10, 11, 12: Protección de rutas.

```
def test_dashboard_sin_sesion_redirige(self):
    """El dashboard sin sesión redirige al login."""
    response = self.client.get('/')
    self.assertRedirects(response, '/login/')

def test_providers_sin_sesion_redirige(self):
    """Providers sin sesión redirige al login."""
    response = self.client.get('/providers/')
    self.assertRedirects(response, '/login/')

def test_transactions_sin_sesion_redirige(self):
    """Transactions sin sesión redirige al login."""
    response = self.client.get('/transactions/')
    self.assertRedirects(response, '/login/')
```

self.assertRedirects(response, '/login/'): verifica que la respuesta es una redirección (HTTP 302) a **/login/**. Sin sesión activa, todas las páginas protegidas deben redirigir al login.

Test 13: Logout sin sesión.

```
def test_logout_sin_sesion_redirige(self):
    """Logout sin sesión redirige al login."""
    response = self.client.get('/logout/')
    self.assertRedirects(response, '/login/')
```

Verifica que el logout también redirige al login cuando no hay sesión.

11.4 Cómo ejecutar los tests.

```
python manage.py test
```

Salida esperada:

```
Found 13 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
-----
Ran 13 tests in 10.732s

OK
Destroying test database for alias 'default'...
```

Cada punto **.** es un test que ha pasado. Si un test falla aparece **F** con el detalle del error.

Para ejecutar solo los tests de una app:

```
python manage.py test core
python manage.py test frontend
```

Para ejecutar un test específico:

```
python manage.py test core.tests.ProviderTests.test_listar_proveedores
```

11.5 El verificador de estilo: flake8.

Además de los tests funcionales, el proyecto usa flake8 para verificar que el código sigue el estándar PEP8:

```
flake8 core/ --exclude=core/migrations/
```

Los errores más comunes que detecta:

- **E501**: línea demasiado larga (más de 79 caracteres).
- **W292**: falta salto de línea al final del archivo.
- **F401**: import no utilizado.
- **E302**: faltan líneas en blanco entre funciones.

Si no aparece ninguna salida el código es correcto. Las migraciones se excluyen porque son archivos generados automáticamente por Django y no siguen estrictamente PEP8.

12. Git y el control de versiones.

Git es el sistema que registra todos los cambios del proyecto a lo largo del tiempo. Es como una máquina del tiempo para el código: puedes ver qué cambió, cuándo y por qué, y volver a cualquier punto anterior si algo sale mal.

12.1 ¿Qué es Git y por qué es importante?

Imagina que estás escribiendo un documento de Word importante. Cada cierto tiempo guardas una copia: "informe_v1.docx", "informe_v2.docx", "informe_final.docx", "informe_final_definitivo.docx"... Este sistema manual es caótico y ocupa mucho espacio.

Git hace esto automáticamente y de forma mucho más inteligente:

- Guarda solo los cambios (no copias completas).
- Registra quién hizo cada cambio y cuándo.
- Permite trabajar en equipo sin pisar el trabajo de otros.
- Permite volver a cualquier versión anterior.
- Permite trabajar en ramas paralelas y luego unirlos.

GitHub es un servicio web que aloja repositorios Git en la nube. Es donde está el código de este proyecto: <https://github.com/SalvaIES/payment-gateway-api>

12.2 Conceptos fundamentales de Git.

Repositorio (repo): la carpeta del proyecto con todo su historial de cambios. Internamente es la carpeta oculta `.git/` que Git crea automáticamente.

Commit: una "fotografía" del código en un momento dado. Cada commit tiene un mensaje descriptivo y un identificador único (hash). Es la unidad básica del historial.

Branch (rama): una línea de desarrollo independiente. En este proyecto solo usamos la rama `main` (la principal).

Remote: una copia del repositorio en otro lugar, generalmente en GitHub. Se llama `origin` por convención.

Push: enviar los commits locales al remote (GitHub).

Pull: descargar los commits del remote al ordenador local.

Clone: descargar una copia completa del repositorio por primera vez.

12.3 El archivo **.gitignore**.

El archivo **.gitignore** lista los archivos y carpetas que Git debe ignorar completamente:

# Python	
__pycache__/	← archivos compilados de Python (se generan automáticamente)
*.py[cod]	← archivos .pyc, .pyo, .pyd
*.pyo	
*.pyd	
# Virtual environment	
venv/	← el entorno virtual (cada usuario tiene el suyo)
env/	
.env	← las claves secretas (NUNCA en GitHub)
# Django	
*.log	← archivos de log
local_settings.py	← configuración local
db.sqlite3	← la base de datos local (cada usuario tiene la suya)
media/	← archivos subidos por usuarios
# IDE	
.vscode/	← configuración de VS Code
.idea/	← configuración de PyCharm
*.swp	← archivos temporales de Vim
# OS	
.DS_Store	← archivos del sistema de macOS
Thumbs.db	← archivos del sistema de Windows

Los tres más importantes son:

- **.env**: las claves secretas nunca deben estar en GitHub.
- **venv/**: el entorno virtual es local y pesa mucho.
- **db.sqlite3**: la base de datos es local, cada desarrollador tiene la suya.

12.4 El flujo de trabajo del proyecto.

El flujo que hemos seguido en este proyecto:

1. Hacer cambios en el código.

Editar archivos, crear nuevas funcionalidades, corregir errores.

2. Preparar los cambios para el commit.

```
git add .
```

`git add .` añade todos los archivos modificados al "staging area": una zona de preparación antes del commit. El punto `.` significa "todo". Si solo quisieras añadir un archivo concreto sería `git add core/views.py`.

3. Crear el commit.

```
git commit -m "feat: integración completa con Stripe"
```

El mensaje del commit describe qué cambios contiene. En este proyecto hemos seguido una convención:

- `feat::` nueva funcionalidad .
- `fix::` corrección de error.
- `docs::` cambios en documentación.
- `hito 1:`, `hito 2::` entregas por hito del proyecto.

4. Subir los cambios a GitHub.

```
git remote set-url origin https://SalvaIES:TOKEN@github.com/SalvaIES/payment-gateway-api.git
git push origin main
git remote set-url origin https://github.com/SalvaIES/payment-gateway-api.git
```

`git push origin main` envía los commits de la rama `main` al repositorio remoto `origin` (GitHub). Primero ponemos el token en la URL para autenticarnos y luego lo quitamos por seguridad.

12.5 Autenticación con GitHub: Personal Access Token.

GitHub ya no acepta contraseñas normales para operaciones Git. En su lugar usa **Personal Access Tokens** (PAT).

Cómo crear un token:

1. Ve a github.com → Settings → Developer settings → Personal access tokens → Tokens (classic).
2. Click en "Generate new token (classic)".
3. Nombre: cualquiera (ej. **payment-gateway-api**).
4. Expiración: 90 días.
5. Permisos: marca **repo**.
6. Click en "Generate token".
7. **Copia el token:** solo se muestra una vez.

El token empieza por **ghp_** y tiene este aspecto:

ghp_pvYZ1vIgR4fwsow3BHDWmNLpKPA89U0RUHHm

Se usa en la URL del remote:

```
git remote set-url origin https://USUARIO:TOKEN@github.com/USUARIO/REPOSITORIO.git
```

Después del push siempre limpiamos el token de la URL:

```
git remote set-url origin https://github.com/SalvaIES/payment-gateway-api.git
```

12.6 El historial de commits del proyecto.

Cada hito del proyecto tiene sus commits correspondientes. Se puede ver el historial con:

```
git log --oneline
```

El historial del proyecto refleja el progreso real del desarrollo:

```
docs: memoria técnica y manual actualizados con Stripe, Redsys y PayPal
feat: integración completa con PayPal sandbox
feat: botones de pago Stripe y Redsys en el frontend
feat: integración completa con Redsys + webhooks
feat: integración completa con Stripe: PaymentIntent y webhooks
feat: ordenación por columnas en la tabla de transacciones
feat: gráfico de transacciones por estado en el dashboard
feat: paginación en la lista de transacciones y botones eliminar
feat: marcar incidencias desde el frontend
fix: mejorar contraste de títulos en el dashboard
frontend: dashboard, proveedores y transacciones con Django Templates
hito 4: memoria técnica, manual de usuario y correcciones PEP8
hito 3: tests unitarios para proveedores y transacciones
hito 2: modelos, serializers, vistas y rutas de la API
hito 1: ERD, product backlog y README
```

12.7 Trabajar en múltiples equipos.

Este proyecto se desarrolló en varios ordenadores (Linux del instituto, Windows de casa, Linux de casa). El flujo para sincronizar el trabajo entre equipos es:

Al llegar a un equipo nuevo:

```
git pull origin main          # descargar los últimos cambios
pip install -r requirements.txt # instalar nuevas librerías si las hay
python manage.py migrate      # aplicar nuevas migraciones si las hay
python manage.py runserver     # arrancar el servidor
```

git pull descarga los commits que hay en GitHub y los aplica localmente. Si alguien más hizo cambios mientras trabajabas en otro equipo, **pull** los trae.

Importante: el archivo **.env** no está en GitHub, así que cada equipo tiene el suyo propio. Si añades variables nuevas al **.env** (como cuando añadimos PayPal), hay que añadirlas manualmente en cada equipo.

12.8 Comandos Git más usados en el proyecto.

<code>git status</code>	<code># ver qué archivos han cambiado</code>
<code>git diff</code>	<code># ver exactamente qué cambió en cada archivo</code>
<code>git log --oneline</code>	<code># ver el historial de commits resumido</code>
<code>git pull origin main</code>	<code># descargar cambios del servidor</code>
<code>git add .</code>	<code># preparar todos los cambios</code>
<code>git commit -m "mensaje"</code>	<code># crear un commit</code>
<code>git push origin main</code>	<code># subir cambios al servidor</code>
<code>git remote -v</code>	<code># ver la URL del repositorio remoto</code>

12.9 Por qué el requirements.txt es flexible.

El `requirements.txt` del proyecto usa `>=` en lugar de `==`:

```
Django>=5.2
django-rest-framework
python-dotenv
requests
sqlparse
stripe
pyresys
paypal-rest-sdk
flake8
tzdata
```

Esto es deliberado. El proyecto se desarrolló en dos sistemas:

- **Linux con Python 3.12:** puede instalar Django 6.0.4.
- **Linux con Python 3.10:** solo puede instalar Django 5.2 porque Django 6 requiere Python 3.12.

Con `Django>=5.2`, `pip` instala la versión más reciente compatible con el Python disponible en cada sistema. Así el mismo `requirements.txt` funciona en ambos entornos.

En un proyecto de producción real se usaría `==` con versiones exactas para garantizar que todos los entornos son idénticos. Pero en este proyecto educativo la flexibilidad es más práctica.

13. Resumen final.

Este proyecto es una API REST completa para la gestión centralizada de pasarelas de pago, desarrollada con Django REST Framework. Aquí está todo lo que se ha construido.

Lo que hace el sistema.

Actúa como intermediario entre una plataforma de comercio electrónico y tres pasarelas de pago. Registra cada intento de pago, lo procesa a través de la pasarela correspondiente, recibe notificaciones automáticas del resultado y actualiza el estado. Todo accesible desde un panel web visual o directamente desde la API.

Cómo está construido.

- **La API REST** recibe peticiones HTTP y devuelve JSON. Está protegida con autenticación por token: sin token no se puede acceder a ningún endpoint. Los datos se validan antes de guardarse gracias a los serializers.
- **La base de datos** tiene dos tablas: **Provider** (las pasarelas configuradas) y **Transaction** (cada intento de pago). La relación es uno a muchos: un proveedor tiene muchas transacciones.
- **El frontend** es un panel web con Django Templates. No accede directamente a la base de datos sino que llama a la propia API. Tiene login con sesión, dashboard con estadísticas y gráfico, gestión completa de proveedores y transacciones con filtros, paginación y ordenación.

Las tres integraciones de pago.

- **Stripe:** al crear una transacción se genera automáticamente un PaymentIntent. El usuario paga con tarjeta directamente en nuestra web. Stripe notifica el resultado via webhook y el sistema actualiza el estado automáticamente.
- **Redsys:** genera un formulario firmado con HMAC-SHA256 y redirige al usuario al TPV virtual del banco. El banco procesa el pago y Redsys notifica el resultado via webhook. Requiere ngrok en desarrollo para recibir las notificaciones.
- **PayPal:** redirige al usuario a la web de PayPal para aprobar el pago. Cuando el usuario vuelve, el sistema ejecuta el cobro definitivo via API y actualiza el estado.

Las herramientas de seguridad.

- Claves secretas en **.env** nunca en GitHub.
- Autenticación por token en todos los endpoints.
- Verificación de firma en webhooks de Stripe y Redsys.
- Protección CSRF en todos los formularios.
- **on_delete=PROTECT** para no perder el historial de transacciones.

Los números del proyecto.

- **2** modelos en la base de datos.
- **3** pasarelas de pago integradas.
- **13** tests automáticos pasando.
- **10** endpoints en la API REST.
- **5** páginas en el frontend.
- **15+** commits en GitHub con trazabilidad del progreso.

El proyecto cumple y supera todos los requisitos del enunciado: gestión de proveedores, registro de transacciones, gestión de incidencias, autenticación, historial filtrado, Django REST Framework, SQLite, PEP8, tests y documentación. Además añade integración real con las tres pasarelas de pago y un frontend completo.